

Watheeq Legal Suite

Features & Capabilities

Clario360 · Watheeq (internal/lex)

Code-verified capability reference

Prepared for executive & product review

Table of Contents

(Right-click → Update Field in Word to populate page numbers.)

1. Executive Overview

1.1 What Watheeq Is

Watheeq is a sovereign, Saudi-native legal operations platform — the legal suite of the Clario360 platform. It gives a corporate legal department a single, governed system of record for the entire legal lifecycle: intake of legal service requests, litigation and case management, investigations, legal consultations and advisory, contract lifecycle management, litigation hearings and judgments, settlements and alternative dispute resolution, delegated approvals, and the qualified electronic signature of legal instruments — all under Saudi data-residency and records-retention rules and all available in Arabic and English.

Watheeq is not a re-skinned generic case tracker. It is a purpose-built legal operating system whose data model, workflow engine, role matrix, and government integrations were designed against the working practices of a large Saudi legal department. The reference deployment is modelled on **Abdullah Al Othaim Investment Company** — a multi-business-unit group whose legal function serves many requesting departments, escalates work through a defined organizational hierarchy, and must produce evidence-grade records that stand up in Saudi courts and before regulators.

The design goal is simple to state and hard to achieve: **every legal action a department takes should be initiated, routed, approved, executed, and archived inside one governed system, with a tamper-evident record of who did what, when, under whose authority, and where the data lives.**

1.2 Positioning — the Sovereign, Integrated, Arabic-Native Wedge

The legal-technology market is crowded with capable but foreign-hosted contract and matter tools. Watheeq's differentiation is deliberate and defensible on three axes that generic products cannot easily match:

- **Sovereign by construction.** Data residency is enforced in code, not promised in a data sheet. The platform refuses to write legal records or archives to storage regions outside the Kingdom when in-Kingdom residency is required, and it fails closed — an out-of-Kingdom target is rejected before any bytes are written, not logged after the fact. Deployment can be SaaS (in-Kingdom multi-tenant), on-premise / VPC in the customer's own data centre, or fully air-gapped for the most sensitive government and defence engagements.
- **Integrated with the Saudi legal ecosystem.** Watheeq speaks to the systems a Saudi legal department actually uses: **Najiz** (the Ministry of Justice court portal, via the Takamul integration surface) for hearing and case synchronization and for issuing powers of attorney (wakala); **Nafath** (the Saudi national single-sign-on identity service) for identity confirmation of signatories; and **emdha** (an NCDC-licensed Trust Service Provider) for qualified electronic signatures under the Saudi Electronic Transactions Law. These government rails are built and hardened, and are honestly presented as "sovereign-ready, pending government onboarding" — they run against sandbox transports until live credentials are provisioned, and they never fabricate a successful government connection they do not have.
- **Arabic-native, not Arabic-translated.** Every role, every service-catalog step, every workflow form field, and every screen carries first-class Arabic alongside English, with right-to-left layout

as the default. Dates render in the official Umm al-Qura Hijri calendar alongside Gregorian; numbers render in Arabic-Indic digits; currency renders in Saudi Riyal; deadlines are computed against a Saudi working-week and public-holiday profile with a Ramadan working-hours overlay. This is engineered into a single formatting layer that every module consumes, so bilingual, RTL-correct, Hijri-aware output is the norm rather than a per-screen afterthought.

The wedge, in one sentence: **WatheeQ is the legal platform a Saudi general counsel can adopt without exporting the organization's most sensitive records offshore, without abandoning Arabic, and without bolting the Saudi courts and identity rails on from outside.**

1.3 Suite Architecture at a Glance

WatheeQ is delivered as the lex service within the Clario360 platform, and it deliberately reuses hardened platform infrastructure rather than reinventing it:

- **A dedicated legal service** exposes the full legal API. Every legal capability is reachable under two equivalent route prefixes, `/api/v1/lex/...` and `/api/v1/watheeQ/...`, so the product can be branded and integrated as "WatheeQ" while sharing one implementation.
- **A shared workflow and approval engine** — a real, governed finite-state-machine engine — drives every approval chain, human task, and multi-step legal process. WatheeQ does not embed its own bespoke workflow logic; it consumes the platform engine through a subject-agnostic approval orchestrator, which means cases, requests, consultations, investigations, settlements, and contract sign-offs all ride the same battle-tested execution core.
- **A connector / integration framework** provides a uniform way to wire the department into enterprise and government systems — single sign-on, HR/identity provisioning, e-archiving, email intake and dispatch, e-signature, and the Saudi government rails — each with health grading, secret custody, and honest sandbox-versus-production status.
- **Platform services** provide identity and authentication, tenant isolation, immutable audit, notifications, file storage, and licensing. WatheeQ inherits these, so a legal-only deployment still stands on a production-grade foundation.
- **A suite-specific legal workspace** — the WatheeQ front end — renders the legal-department experience: role-aware navigation, persona switching, and the domain screens for every legal function, all localized.

The division of labour is intentional: **the legal domain logic is WatheeQ's; the execution, identity, audit, and integration plumbing is shared, proven platform infrastructure.** This is why a WatheeQ-only tenant is materially lighter to run than the full platform while still benefiting from enterprise-grade governance.

1.4 Who Uses WatheeQ

WatheeQ is built around the real organizational shape of a corporate legal department. It recognizes a fourteen-role legal hierarchy spanning the requesting business (employees, department managers, business-unit CEOs, the CEO), the legal function itself (legal director, section managers for cases and for contracts, supervisors, handling officers/lawyers, and advisors), an oversight layer (a shared-services manager and a compliance auditor), and a system administrator. Each role has its

own landing experience, its own scoped navigation, and its own precisely bounded set of permissions — a legal officer who drafts a case cannot approve it; an auditor can see everything and change nothing; a system administrator can configure the platform but cannot make legal decisions. Chapter 17 covers this role model in full; it is introduced here because the role hierarchy is the backbone around which every capability in this document is organized.

1.5 The Competitive Position

It is worth being explicit about how Watheeq differs from the alternatives a Saudi general counsel is likely to be shown, because the differences are structural rather than cosmetic.

Versus foreign-hosted legal and contract platforms. The capable international products in this space are, almost without exception, hosted outside the Kingdom, built English-first with Arabic as a translation layer, and integrated with Western e-signature and identity providers rather than the Saudi rails. For an organization whose legal records include privileged and personal data subject to the Kingdom's data-protection and records-retention rules, adopting such a product means either exporting those records offshore or negotiating a bespoke in-region hosting arrangement that the product was not designed for. Watheeq inverts these defaults: in-Kingdom residency is the enforced default, Arabic and RTL are native, and the Saudi courts, identity, and trust-service rails are first-class integrations. The organization does not have to fight the product's assumptions to comply with Saudi law.

Versus generic case or matter-management tools. Many organizations run their legal function on a general-purpose tracker — a configured instance of a workflow or ticketing tool. These handle the "list of matters" but not the governance: they rarely encode a fourteen-role segregation-of-duties matrix, cryptographic delegation-of-authority on approvals, immutable records with legal hold, qualified electronic signatures, or Saudi-calendar SLA escalation. Watheeq is not a configured tracker; it is a legal operating system in which these governance controls are the substance of the product, not something an implementation team must assemble from primitives.

Versus building it in-house. The alternative some large organizations consider is building on top of generic infrastructure. The cost of doing so well — a governed workflow engine, a tamper-evident audit trail, residency enforcement, the government integrations, the role and approval model — is exactly the cost Watheeq has already absorbed, and the honesty about what is live versus sandbox reflects a product that has done the hard integration work rather than one still promising it.

The net competitive position is that Watheeq occupies a space its rivals largely do not: a **sovereign, Arabic-native, deeply-governed legal operating system built for the Saudi legal ecosystem**, delivered as a product rather than a project. For a general counsel, that translates into a shorter path to a compliant, defensible legal function — and for the organization, into legal records that never had to leave the Kingdom to be managed well.

2. The Legal Operations Lifecycle

2.1 One Journey, Many Aggregates

Watheeq models legal work as a lifecycle that begins with a request for legal service and ends with a delivered, archived outcome — but it does so through a **hub-and-spoke architecture** rather than a single monolithic record. At the centre is the **legal request spine**: a canonical request row that every legal affairs service references. Around it sit first-class domain aggregates — cases, consultations, investigations, settlements, matters, contracts — each with its own lifecycle, each linking back to the spine.

This matters for two reasons. First, it means the system has a single, auditable front door: however a piece of legal work enters, it is represented on the spine, so nothing is untracked. Second, it means each legal domain can evolve, deploy, and be reasoned about independently, because the links between them are deliberately loose. The result is a platform that presents as one coherent legal journey to the user while remaining modular and independently governable underneath.

The end-to-end journey, at the level a legal director cares about, runs:

Intake → Submit → Approve → Route → Execute → Monitor → Deliver / Close.

2.2 The Stages in Business Terms

Intake. Legal work enters through one of two channels: a person submits a request directly in the platform, or a request arrives by email into a monitored legal mailbox and is classified automatically. Either way, the request is validated (is the requester eligible for this service? is the stated urgency justified?) and recorded on the spine as a draft.

Submit. The requester submits the draft. Depending on the service, the request may require a requester-side approval (for example, the requesting department manager's sign-off under a delegation-of-authority matrix), a provider-side approval (the legal department's acceptance), both, or neither. Services that require no approvals are auto-approved and routed immediately.

Approve. Where approvals are required, they run through the shared approval engine: approval tasks are created for the right approvers, decisions are recorded with full audit and — where the service demands it — cryptographic delegation-of-authority evidence, and the request advances only when the required quorum is met.

Route. On approval, the request is routed. This is the moment the spine decides what kind of legal work this is and, where appropriate, materializes the right downstream aggregate: a litigation request becomes a legal case; a request for an opinion or consultation becomes a consultation; other services execute directly on the spine. Routing is idempotent — a request is never double-spawned.

Execute. The legal department confirms the request is complete and begins work. This starts the service-level-agreement clock: acknowledgement deadlines, turnaround deadlines, and escalation deadlines are all computed against the Saudi working calendar. Substantive changes re-open the completeness gate so the clock restarts only on a fresh confirmation.

Monitor. While work is in flight, the platform watches the SLA clock. As deadlines approach and pass, it acknowledges, breaches, and escalates — first to the section supervisor, then to the department manager, then to the shared services manager — emitting notifications through email, calendar, and in-app channels, with deduplication so the same event is never announced twice.

Deliver / Close. Work is delivered and the request is closed. Return and cancellation are the off-ramps at every stage. Throughout, every material transition writes an append-only audit record and relays it to the platform's immutable, hash-chained audit ledger, and emits a domain event for any out-of-process subscriber.

2.3 The Eight Legal Services

Watheeq ships, seeded for every tenant, an eight-service legal catalog that defines what the legal department offers and how each service is routed. Each service declares its channel (platform, email, or both) and whether it needs requester-side and/or provider-side approval:

#	Legal service	Requester approval	Provider approval	Downstream on routing
1	Legal Consultation	No	Yes	Consultation
2	Contract Review	Yes	Yes	Executes on the spine / contract review desk
3	Contract Drafting	Yes	Yes	Executes on the spine
4	Litigation Support	Yes	Yes	Legal Case
5	Legal Opinion	No	Yes	Consultation
6	Regulatory Compliance	No	Yes	Executes on the spine
7	Power of Attorney	Yes	Yes	Executes on the spine
8	General Legal Request	No	No	Auto-approves and routes

Two service families automatically materialize a downstream aggregate — litigation requests become cases, and consultation/opinion requests become consultations. Investigations and settlements are created directly through their own front doors (an investigation is opened by the cases section; a settlement is opened against an existing legal matter). This is a considered design: it keeps the routing model simple and predictable for the services that most benefit from automation while allowing the more specialized workflows to be initiated deliberately.

2.4 The Request State Machine, Step by Step

Because the request is the backbone of the whole platform, it is worth setting out its state model explicitly — this is the contract that lets a legal director trust what a dashboard tells them. A request moves through an explicit set of states, and only legal transitions are permitted; an illegal move is rejected rather than silently accepted.

State	Meaning	Where it can go next
Draft	Created but not yet submitted	Submitted, Cancelled
Submitted	Submitted; awaiting the required approvals	Pending requester approval, Pending provider approval, Approved, Returned, Cancelled

Pending requester approval	Awaiting the requesting side's delegation of authority sign-off	Pending provider approval, Approved, Returned, Cancelled
Pending provider approval	Awaiting the legal department's acceptance	Approved, Returned, Cancelled
Approved	Authorized; ready to be routed	Routed, Returned, Cancelled
Routed	Routed to its downstream domain (or executing on the spine)	In execution, Returned, Cancelled
In execution	Completeness confirmed; the SLA clock is running	Delivered, Returned, Cancelled
Delivered	Work delivered to the requester	Closed, Returned
Closed	Complete — terminal	—
Returned	Sent back for correction	Submitted, Cancelled
Cancelled	Abandoned — terminal	—

Three properties of this model matter to the business. First, the **approval branch adapts to the service**: a service requiring no approvals moves from submitted straight to approved and is routed automatically, while a service requiring both a requester-side and a provider-side approval passes through both pending states — so the pipeline is exactly as heavy as the governance of that service requires, and no heavier. Second, **return and cancellation are available at every active stage**, so there is always a defined, audited way to send work back or stop it, rather than an ad-hoc workaround. Third, **the terminal states are genuinely terminal** — a closed or cancelled request has no outgoing transition — so "closed" means closed, and cannot be quietly re-opened without a new, audited action. Every one of these transitions is guarded by optimistic concurrency, so two users acting on the same request at once cannot corrupt its state; the second action is rejected with a conflict rather than silently overwriting the first.

2.5 Why the Single-Record-Yet-Modular Design Matters to the Business

For a general counsel, the practical payoff of this architecture is threefold. **Nothing falls through the cracks** — every request is on the spine, with a status, an owner, and an SLA. **Every downstream artifact traces home** — a case, a consultation, or a settlement always knows the request that spawned it, and the request always knows what it became. And **the department can adopt the platform incrementally** — because the domains are loosely coupled, an organization can start with, say, requests and consultations and add litigation, contracts, and settlements over time without a re-platforming exercise.

3. Matter & Case Management

3.1 The Legal Request Spine as the System of Record

The legal request is the atomic unit of legal work in Watheeq. Every request carries a human-readable request number, a service, a beneficiary (the organizational entity the work is for), a priority, a status, and a full audit trail. The request's lifecycle is governed by an explicit state machine so that only legal transitions are possible — a request cannot jump from draft straight to closed, and any illegal move is rejected rather than silently accepted.

The request status set is precise and business-meaningful: **draft, submitted, pending requester approval, pending provider approval, approved, routed, in execution, delivered, closed, returned, cancelled**. Closed and cancelled are terminal. This is not incidental detail — it is the contract that lets a legal director trust the pipeline. When a dashboard says a request is "in execution," that means the completeness gate was passed, the SLA clock is running, and the work is genuinely under way; it cannot mean anything else.

Requests carry a two-tier urgency — **urgent** or **normal** — which is deliberately simpler than the four-tier priority used inside cases and matters, because the request front door needs a fast, unambiguous urgency signal, while the internal legal work needs finer prioritization. The platform maps between the two so that an urgent request lands as a high-priority case.

A notable governance feature sits right at intake: **urgency justification**. When a requester marks a request urgent, the system requires a substantive justification and actively rejects non-reasons — "forgot," "last minute," "asap," and their Arabic equivalents are refused. This is enforced at the data layer, not merely in the UI, so the discipline cannot be bypassed by calling the API directly. It is a small feature with a large cultural effect: it stops "everything is urgent" from eroding the SLA model.

3.2 Intake — Two Front Doors, One Spine

Platform intake. An authenticated user submits a request against a catalog service. The system verifies the service is active and accepts platform submissions, evaluates the requester's eligibility for that service against the organizational registry, and — on success — creates the request on the spine, inheriting the service's request type and its requester/provider approval requirements. Eligibility failures are refused with a clear reason and counted, so the department can see where access rules are biting.

Email intake. Legal work also arrives by email, and Watheeq treats this as a first-class, governed channel rather than a manual copy-paste chore. An inbound email hitting the legal mailbox webhook is authenticated by HMAC signature over its timestamp and body, with a replay-tolerance window; it is de-duplicated by message identifier so a redelivered email never creates a second request; its body and attachments are persisted; and it is classified against the active catalog to resolve the service, request type, and beneficiary before a routed request is created. Crucially, email intake never writes to the request table directly — it always goes through the same request-creation path as platform intake, so both channels are subject to identical validation and audit.

Whichever door it comes through, the request is now a first-class, tracked, audited object on the spine.

3.3 Litigation Case Management — a Two-Phase Intake

Litigation is the most governance-intensive legal workflow, and Watheeq models it with a two-phase case intake that mirrors how a real legal department commissions and staffs a case.

A legal case carries its own status set — **intake, phase 1, phase 2, open, under procedure, closed, cancelled** — plus supporting attributes: which side the company is on (**plaintiff** or **defendant**), a litigation-strength assessment (**strong** or **weak**) recorded at intake, and the standard priority.

Phase 1 — administrative directive and authority. Opening a case is not a clerical act; it is an authorized decision. Phase 1 runs an approval chain up the organizational hierarchy, from the legal director to the CEO, and it requires the evidence a real authorization needs: a reference to the CEO's directive to commence legal action, a delegation-of-authority reference, and the strength assessment. The approval chain is sequential and requires unanimity, and it is validated with cryptographic delegation-of-authority evidence tied to the approving authority — so the record does not merely say "the CEO approved," it can prove which key, bound to which authority limit, signed the approval. (Chapter 10 covers this cryptographic authority model in depth.)

Phase 2 — assignment and hand-off. With the directive approved, the case moves to a legal-director-to-section-manager hand-off: the work is estimated, and the section manager, a supervisor, and a handling officer are assigned. This is the moment the SLA clock is stamped, and the case moves to open, ready to be worked. From open, the case supports the full apparatus of litigation management — parties, hearings, tasks, comments, documents, status changes, strength and priority updates, and dedicated, separately-authorized assignment actions (transfer to a section manager, assign a supervisor, assign an officer).

The separation of these phases is not bureaucracy for its own sake. It encodes a control principle: **the decision to litigate and the authority to litigate are captured, evidenced, and approved before a single billable hour of case work is assigned.** For a general counsel and an auditor, this is exactly the trail they need.

3.4 Matters — the General Container for Legal Work

Not all legal work is a litigation case. A **matter** is Watheeq's general legal-work container, with its own lifecycle — **intake, open, in review, waiting on business, on hold, closed, cancelled** — and its own sub-resources: comments, documents, related links, obligations, linked contracts, and an audit trail. Matters support conflict checking at creation and a triage step, and they are the anchor to which settlements attach. Where a case is the specialized vehicle for litigation, a matter is the flexible vehicle for advisory work, corporate legal support, regulatory workstreams, and anything that does not fit a more specific aggregate.

3.5 Classification, the Organizational Registry, and the Working Calendar

Three supporting capabilities make case and matter management coherent across a large organization.

Case classification provides a maintainable taxonomy — a tree of case types with codes, usage tracking, cascade and merge operations, reordering, and its own audit — so that reporting and routing rest on a controlled vocabulary rather than free text.

The organizational registry is Watheeq's model of the customer's own structure: the business units, departments, and entities that raise requests, benefit from services, and sit in the escalation ladder. Registry entries carry role bindings and an escalation resolution, so the platform can answer "who is the section supervisor for this entity, and who does an overdue item escalate to" from live organizational data rather than a hard-coded chart. The registry is deliberately referenced by opaque identifier from the request spine, which is what lets the request module ship and operate even where the registry is still being populated.

The working calendar encodes Saudi working time: the working week, public holidays (both fixed Gregorian national days and Hijri religious holidays), and a Ramadan overlay. Every SLA deadline, acknowledgement window, and escalation rung is computed against this calendar in the Kingdom's timezone, so "two working days" means two Saudi working days — not two calendar days, and not two working days on some foreign business calendar.

3.6 Eligibility and the Discipline of Intake

Before a request is even accepted, Watheeq applies two intake disciplines that protect the integrity of the pipeline. The first is **eligibility**: each catalog service can declare who is entitled to raise it — all employees, a specific department, a specific role, or those holding a position in the delegation of authority matrix — and a request that fails the eligibility check is refused with a clear reason and counted, so the department can see where its access rules are biting and whether they are set correctly. The second is the **urgency justification** discipline described earlier: marking a request urgent requires a substantive justification, and the platform actively refuses non-reasons at the data layer, so the urgency signal that drives the fastest SLAs cannot be casually abused. Together these ensure that what enters the pipeline is a legitimate, properly-classified request from an entitled requester — the precondition for every downstream guarantee.

The two-tier request urgency (urgent or normal) maps deterministically onto the four-tier internal priority used by cases, consultations, and matters — urgent becomes high, normal becomes medium — so that the fast, unambiguous urgency the request front door needs translates cleanly into the finer prioritization the internal legal work needs, without the two models drifting apart.

3.7 Service Levels, Acknowledgement, and Escalation

Case and matter management is only as trustworthy as its deadlines, and Watheeq treats service-level agreements as a governed, first-class subsystem rather than a cosmetic timer. Administrators maintain SLA **targets** per service and priority. Acknowledgement is expressed in the unit that matches the urgency: urgent work is acknowledged in **working hours** (the default is four), and normal work in **working days** (the default is one) — and the platform enforces the pairing, so an urgent target cannot be misconfigured in days. Turnaround targets are seeded per service at the conservative end of the documented ranges — for example, contract review at three working days for urgent and five for normal; legal consultation at four and six; contract drafting at five and eight — and are fully administrator-maintainable.

Each request gets a **single SLA clock**, created idempotently when execution begins, which materializes concrete acknowledgement, turnaround, and escalation-level deadlines from the working calendar and tracks an outcome of **pending, on time, or breached**. On top of the turnaround deadline sits a fixed, database-enforced **escalation ladder** measured in working days after a breach — the first rung at two working days, the second at four, the third at six — so that a breached item does not merely turn red but is actively driven up the organization.

A monitor process advances each clock through its lifecycle: it fires acknowledgement-due reminders, marks breaches, and advances the escalation level, resolving each rung to the right organizational role from the live registry — level one to the section supervisor, level two to the department manager, level three to the shared-services manager. It enqueues the resulting notifications — acknowledgement-due, breach, and escalation — on an **append-only outbox** across email, calendar, and in-app channels, with a de-duplication key on the clock, event type, and escalation level so that a given clock never double-emits the same event at the same level. Every reminder, breach, and escalation is therefore both delivered and recorded. The effect for management is a legal pipeline that escalates itself, on the Kingdom's own working calendar, with an auditable trail of every reminder and breach — the department does not have to remember to chase overdue work, because the platform chases it, up the correct chain, on the correct dates.

3.8 The Complete Case Toolkit

Once a case is open, Watheeq gives the handling team the full apparatus of litigation management as governed sub-resources of the case, each audited and permission-scoped: **parties** (with a company-side designation of plaintiff or defendant), **hearings**, **tasks** with estimates and assignment, **comments**, and **document** links — all addable individually or in bulk. Distinct, separately-authorized **assignment** actions transfer a case to a section manager, assign a supervisor, and assign an officer; these use the restricted assign verb with no coarse fallback, so allocation is a controlled act rather than an incidental edit. Case attributes — status, litigation strength, and priority — are updated through their own governed actions, each writing to the case audit trail. Alongside the case-specific apparatus, the litigation module adds pleadings, hearing reports, experts, judgments (with idempotency so a judgment is never double-recorded), and defendant-response handling, with the sensitive workflow decisions — pleading approvals and response reviews — gated on the case-approve authority and dynamic segregation of duties, so the author of a pleading cannot approve their own. The result is that a case is not a folder of attachments but a fully-structured, governed litigation record.

3.9 What a Legal Director Gets From This Chapter's Capabilities

Taken together, the request spine, the two-phase case intake, the matter container, the organizational registry, the working calendar, and the SLA engine deliver something a corporate legal department rarely has: **a single, governed, self-escalating pipeline in which every piece of legal work is initiated through a controlled front door, authorized before it is staffed, tracked against Saudi-calendar deadlines, escalated automatically when it slips, and recorded immutably from cradle to close**. Everything that follows in this document — investigations, consultations, contracts, litigation detail, settlements, approvals, AI assistance, e-signature, analytics, and the integration platform — hangs off this backbone.

4. Investigations Management

4.1 Overview and Business Purpose

Legal investigations sit at the sensitive intersection of the legal department's remit and the wider governance obligations of the organisation. When a suspected violation, misconduct allegation, policy breach, or compliance concern is raised — whether it originates from a whistle-blower channel, a line manager, an internal audit finding, or a regulator's enquiry — the legal function is frequently asked to conduct or supervise a structured investigation. That process must be defensible: it has to establish the facts impartially, protect the rights and privacy of everyone involved, preserve evidence in a way that would survive later scrutiny, and produce findings and recommendations that decision-makers can act upon with confidence.

The Watheeq **Investigations Management** module is the first-class workspace for exactly this work. It treats each investigation as a governed record with its own lifecycle, its own participants, its own evidence register, its own testimony log, and its own approval chain for signing off the results. Nothing about an investigation is captured as free-form notes bolted onto some other record; the investigation is an aggregate in its own right, with a defined state machine, an immutable audit trail, and strict confidentiality controls applied to the personal and sensitive information it inevitably contains.

Critically, the module is designed to operate **independently** of the litigation and case-management machinery while still being able to link to it. An investigation may be launched on its own — for example, a study of an alleged procurement irregularity — long before anyone knows whether a court case will follow. If the facts warrant it, the investigation can be tied to a legal case, and the investigation's findings can become the evidentiary and analytical foundation for litigation or a disciplinary escalation. This "ships independently, links optionally" philosophy means the legal team can start an investigation the moment a concern arises, without first having to create a case shell it may never need.

4.1.1 What an Investigation Records

Each investigation aggregates the following related information:

- **The investigation header** — a unique investigation number, a subject (the matter under study), the lead investigator, a priority, an optional owning department, and an optional link to a legal case.
- **Parties** — every person or entity connected to the investigation, classified by their role (subject, complainant, witness, investigator, expert, or other).
- **Statements** — recorded testimonies taken from deponents, each attributed to who gave it, who took it, and when.
- **Evidence** — a catalogued register of every exhibit, document, and artefact, each linked to the securely stored underlying file and annotated with chain-of-custody information.
- **Findings** — the substantive conclusions of the investigation.
- **Recommendations** — the final proposed course of action arising from the findings.

- **A governance audit trail** — an append-only, immutable log of every meaningful action taken on the investigation.

4.2 The Investigation Lifecycle

Every investigation moves through a controlled, finite set of states. The lifecycle is deliberately enforced by the system rather than left to convention, so that an investigation cannot, for example, be closed with unapproved results or reopened after it has been formally concluded. The states are:

State	Meaning
Registered	The investigation has been opened and given a number, but substantive work has not yet begun.
In Progress	Active investigation: parties are being registered, statements taken, and evidence gathered.
Results Recorded	The findings have been written up and captured on the record.
Pending Approval	The results have been submitted into the sign-off (approval) chain and await a decision.
Approved	The results have been formally approved. This is a terminal state.
Rejected	The results were sent back by an approver; the investigation returns to active work for rework.
Closed	The investigation has been concluded and closed. This is a terminal state.
Cancelled	The investigation was cancelled before conclusion. This is a terminal state.

4.2.1 State Transitions and Their Guardrails

The system distinguishes sharply between **operational** transitions (which the working investigator drives) and **decision** transitions (which require authority and separation of duties).

- **Registered → In Progress** happens automatically the moment substantive work begins — for instance, when the first party is added. The investigator does not have to remember to "start" the investigation; the act of doing investigative work advances the state.
- **In Progress → Results Recorded** occurs when findings are captured.
- **Results Recorded → Pending Approval → Approved / Rejected** are the sign-off transitions. These **cannot** be forced through the ordinary status-change endpoint. Any attempt to move an investigation directly into **pending approval**, **approved**, or **rejected** through the general status route is rejected by the system with a conflict, because those transitions must flow through the dedicated results-approval chain where authority, quorum, and separation-of-duties are enforced.
- **Close** and **Cancel** are separately gated. Closing or cancelling an investigation is a "close-class" action that requires a distinct closing permission, not merely edit rights. A user who can edit an investigation cannot, by that fact alone, close or decide it.

Terminal states are genuinely terminal: once an investigation is **approved**, **closed**, or **cancelled**, the system refuses any further status transition on it. This prevents the quiet re-opening or after-the-fact alteration of a concluded investigation.

4.2.2 Separation of Duties on Close and Decision

Two independent controls protect the integrity of the most consequential transitions:

1. **Elevated permission gating.** A close-class target (closed or cancelled) is re-checked against a dedicated **investigation close** permission, and an approve/decision-class target (approved or rejected) against a dedicated **investigation approve** permission. Neither transition accepts a coarse "write" or "edit" permission as a fallback. In practice this means a caseworker with day-to-day editing rights simply cannot close an investigation or decide its outcome.
2. **Dynamic separation of duties (author ≠ actor).** On a close or decision transition, the system resolves the original author of the investigation and blocks that same person from being the one who closes or decides it. If the author cannot be resolved for any reason, the guard fails **closed** — that is, it denies the action rather than risk letting the author self-approve. This author-versus-actor parity check is applied consistently across the legal suite's sensitive verbs.

Together these controls mean that opening, working, and concluding an investigation are performed by different hands, with the system — not office etiquette — enforcing the split.

4.3 Intake and Registration

An investigation begins with **registration**. The intake payload captures the essentials:

- **Subject** — a description of what is being investigated. This is treated as sensitive and is protected accordingly (see confidentiality, below).
- **Lead investigator** — the individual with primary responsibility. Also treated as sensitive.
- **Priority** — one of **critical**, **high**, **medium**, or **low**. When the requester does not state a priority, the system defaults it to **medium** so that no investigation is left un-triaged.
- **Investigation number** — a unique identifier scoped to the tenant. It may be supplied explicitly or generated by the system, and it is guaranteed unique among live (non-deleted) investigations for the organisation.
- **Case link (optional)** — a loose reference to a legal case. This is deliberately a soft link with no hard database constraint, so an investigation can be registered with no case whatsoever and later associated with one.
- **Department (optional) and metadata** — for organisational routing and any structured attributes a tenant wishes to attach.

Because the module ships independently of litigation, a fresh investigation is entirely self-sufficient at the point of registration. The moment work starts on it, its state advances from **registered** to **in progress** automatically, and every subsequent action is written to the audit trail.

4.4 Parties and Participants

Investigations turn on people, and the module models the participants explicitly. Each **party** is registered with:

- A **role**, drawn from a controlled list: **subject** (the person or entity under investigation), **complainant**, **witness**, **investigator**, **expert**, or **other**.
- A **name**.

- An optional **identifier** (for example, a national ID or employee number) and an optional **contact**. Both of these are personal data and are protected as sensitive information at rest.
- Optional **metadata**.

Parties can be added, amended, and removed while the investigation is mutable. Registering the first party is one of the natural triggers that moves the investigation from **registered** into **in progress**, reflecting the reality that once you are cataloguing who is involved, the investigation is genuinely under way.

Modelling roles precisely matters for confidentiality and for later reporting: it lets the organisation distinguish, say, the **subject** of an investigation (whose rights require particular care) from a **witness** or an **expert**, and it lays the groundwork for role-appropriate handling of statements and disclosures.

4.5 Statements and Testimony

Investigations frequently rest on what people say. The module provides a dedicated **statement** register for recording testimonies. Each statement captures:

- The **deponent** — either linked to a previously registered party, or named directly for a one-off deponent.
- The **statement body** itself — the substance of the testimony. This is among the most sensitive information the module holds and is protected as sensitive data at rest.
- **When** the statement was taken and **by whom** it was taken.
- Optional **metadata**.

Statements are attributed and timestamped, giving the organisation a defensible record of who said what, to whom, and when. Because statements can be linked back to a party row, the system can present a coherent picture of each individual's involvement — their role, their contact details, and everything they have deposed — while keeping the sensitive bodies protected. Statements may be recorded and removed while the investigation remains mutable, and every such action is audited.

4.6 Evidence Handling and Chain of Custody

Evidence is where investigative rigour is most visibly tested, and the module treats it with corresponding care. Evidence is handled as a **two-part construct**:

3. **The binary itself** — the scanned document, photograph, recording, or file — is uploaded to and stored by the platform's secure file service, tagged to the investigation as its owning entity. The actual bytes never sit loosely in the legal database.
4. **The evidence register entry** — a catalogued reference held against the investigation — links to that stored file and records the chain-of-custody metadata around it.

Each evidence entry captures:

- A link to the **stored file**.
- A **title** and a **description**.
- An **evidence type**, so exhibits can be classified.

- **Who collected it** and **when** it was collected — the essential chain-of-custody anchors.
- Optional structured **metadata** for any additional provenance.

This separation of the artefact from its catalogue entry has real benefits. It keeps the legal record lean and queryable while delegating secure storage, access control, and retention of the actual files to the purpose-built file service. It also means the evidence register can carry rich provenance — who handled the exhibit, when, and under what description — independently of the file's own storage lifecycle. Evidence entries can be added and removed while the investigation is mutable, and every addition or removal is recorded in the immutable audit trail, so the history of what was entered into evidence, and when, is itself tamper-evident.

4.7 Findings and Recommendations

Once the fact-gathering is sufficiently advanced, the investigator records the **findings** — the substantive conclusions — which advances the investigation to **results recorded**. The module supports two ways of capturing findings:

- **Verbatim capture**, where the investigator writes the findings directly, and the text is stored as provided.
- **Assisted drafting**, where the investigator supplies a prompt and the system drafts the findings body through the shared drafting engine, including Arabic-language drafting. When assisted drafting is used, the investigation record is flagged to indicate that the body was drafted with assistance, so that the provenance of the text is transparent on the record.

Separately, the module captures the **final recommendations** — the proposed course of action arising from the findings. Recommendations support the same verbatim-or-assisted drafting choice, and the same transparency flag applies.

Both findings and recommendations are treated as sensitive content and protected at rest. Recording either is a first-class, audited event: the audit log captures that results were recorded, and whether the body was assisted-drafted, so the trail distinguishes human-authored from assisted-drafted content.

4.8 Results Approval — The Sign-Off Chain

Investigation findings are consequential, and the module does not let them be self-certified. Sign-off runs through a dedicated **results-approval chain** built on the same shared, subject-agnostic approval engine that governs case and request approvals across the legal suite. Reusing one engine means investigations inherit the identical guarantees — record locking, quorum handling, delegation-of-authority evidence, and event emission — that the rest of the suite relies on, rather than a bespoke, weaker approval path.

The flow is:

5. **Start approval.** From **results recorded**, the investigator opens the approval chain, which advances the investigation to **pending approval**. An approver role is specified; when none is given, the chain defaults to a single legal-director approval. Optional notes accompany the request.

6. **Decide.** An authorised approver — who, by the separation-of-duties rule, cannot be the person who authored the results — records a decision against the approval task.
 - On **approve**, the investigation advances *pending approval → approved*, a terminal, formally signed-off state.
 - On **reject**, the investigation returns to *in progress* for rework, so the findings can be revised and resubmitted.
7. **Task visibility.** The outstanding approval tasks for an investigation can be listed at any time, so approvers and supervisors can see exactly what is awaiting decision.

Every step of this chain is captured in the immutable investigation audit log, and the approval decision — being the true control point — is the transition on which the author-versus-actor separation is most strictly enforced.

4.9 SLA, Deadlines, and Facts Audit

Investigations are time-sensitive, and the module wires them into the same working-day service-level machinery used elsewhere in the suite so that progress can be measured and lapses surfaced. When an investigation is registered, a working-day SLA clock is started, so acknowledgement and response windows and the escalation ladder can fire against it. As the investigation reaches its conclusion, the SLA clock is resolved, giving the organisation a clean, working-calendar-based measurement of how long the investigation took.

Beyond the SLA clock, the module can schedule **deadline reminders** — for example, objection or hearing deadlines — without introducing any new timing infrastructure. It does this by creating a linked obligation that rides the platform's existing reminder mechanism, with configurable lead-time days ahead of the deadline. This reuse means investigation deadlines are surfaced through the same reliable reminder channel the rest of the suite already depends on.

Finally, the module records **duration facts** into the reporting layer as the investigation progresses, again computed against the organisation's frozen working calendar. This feeds the analytics and KPI surfaces with defensible, working-day-based cycle-time data, so that management can answer questions such as how long investigations of a given priority typically take, and whether that is trending in the right direction.

4.10 Escalation to Cases and Litigation

A recurring pattern in legal work is that an investigation, once its facts are established, must be escalated — into a disciplinary process, a regulatory response, or full litigation. The module supports this cleanly through its **optional case link**.

Because an investigation can carry a soft reference to a legal case, an investigation and a case can be associated at any point in the investigation's life: at registration, if it is clear from the outset that litigation is expected; or later, once the findings reveal that a case is warranted. The linkage is intentionally loose — there is no hard constraint forcing a case to exist — precisely so that the investigation can be started and worked without prematurely committing to litigation.

When an investigation is tied to a legal case, that relationship is respected by the wider system's protective controls. In particular, if the linked case is under a legal hold, the investigation participates in the same hold-aware mutability checks, so that records connected to a matter subject to preservation obligations cannot be quietly altered. This tight coupling between investigations, cases, and legal holds is what allows an investigation's evidence and findings to serve as the reliable foundation for downstream litigation, disciplinary action, or regulatory engagement.

4.11 Confidentiality and Data Protection

Investigations, by their nature, concentrate sensitive personal and organisational information: the identity of subjects and complainants, national identifiers, contact details, verbatim testimony, and conclusions that could materially affect the people named in them. The module is built with confidentiality as a first principle.

- **Sensitive fields are protected at rest.** The subject, the lead investigator, party identifiers and contacts, the bodies of statements, and the findings and recommendations are all held as protected, encrypted content rather than as readable data in the database. Someone with raw database access does not thereby gain the substance of the investigation.
- **Strict tenant isolation.** Every investigation record and every sub-record — parties, statements, evidence, and audit entries — is locked to its owning organisation by database-enforced row-level security. There is no path by which one tenant's investigation data becomes visible to another.
- **Immutable governance audit trail.** Every material action — registration, status change, results recorded, approval events — is written to an append-only audit log that permits inserts but no updates or deletions. The trail of what happened, and when, and by whom, cannot be rewritten after the fact.
- **Permission-scoped access.** Viewing, editing, closing, and deciding an investigation are each governed by distinct permissions, so access to the most sensitive actions is confined to appropriately authorised staff.

The net effect is an investigations workspace that a legal department can trust with its most delicate matters: rigorous about the facts, careful with the people involved, defensible in its record-keeping, and disciplined about who may do what.

5. Legal Consultations & Advisory

5.1 Overview and Business Purpose

Not every demand on a legal department is a contract, a lawsuit, or an investigation. A large and continuous part of the work is **advisory**: colleagues across the business ask the legal team questions and need considered answers. What are the risks in this supplier arrangement? How should end-of-service benefits be calculated for a resigning employee? Are we compliant with the latest e-invoicing rules? Can the board pass this resolution the way it is drafted? These consultations are individually smaller than a case, but collectively they are a major share of the department's output and a major driver of how the rest of the business perceives the legal function's responsiveness and value.

The Watheeq **Legal Consultations & Advisory** module gives this advisory work a proper home. It turns what is too often an untracked stream of emails and corridor conversations into a governed workflow: consultations are submitted, classified by subject matter, routed to the right counsel, answered, signed off, and archived — each step tracked, timed against a service-level target, and audited. Along the way, the module supports assisted drafting of first responses and builds a reusable body of answered questions that becomes an institutional knowledge asset.

Consultations may arise in two ways. They can be **submitted directly** by, or on behalf of, a requester who has a legal question. Or they can be **spawned from the legal-request service catalogue** when an opinion- or advisory-type request is routed (described in Chapter 6). Either way, once a consultation exists it follows the same disciplined lifecycle.

5.2 The Consultation Lifecycle

Each consultation moves through a defined, finite lifecycle:

State	Meaning
Submitted	The consultation has been raised and is awaiting triage.
Classified	Its subject-matter type (and, if needed, priority) has been assigned.
Routed	It has been assigned to an advisor to answer.
Responded	The advisor has recorded the advisory response.
Approved	The response has been signed off.
Archived	The completed consultation has been filed into the knowledge base.

The progression is orderly — submit, classify, route, respond, approve, archive — and each transition is a first-class, audited action. This structure ensures a consultation is never simply "answered and forgotten": it is triaged so it reaches the right expert, its answer is reviewed before it is relied upon, and it is then preserved as reusable knowledge.

5.2.1 Subject-Matter Classification

When a consultation is submitted, it can be **classified** by subject matter, which both routes it to the right expertise and organises the growing knowledge base. The supported consultation types are:

- **General**
- **Contractual**

- **Labor**
- **Regulatory**
- **Corporate**
- **Litigation**
- **Intellectual Property**
- **Tax**
- **Other**

A consultation that arrives without a stated type defaults to **general** so it is never left unclassified, and the type can be assigned or changed during the classify step, which advances the consultation from **submitted** to **classified**. Classification is more than a label: it is the primary axis along which advisory demand is analysed, along which counsel is assigned, and along which answered questions are later found and reused.

5.3 Intake — Raising a Consultation

Submitting a consultation captures:

- A **bilingual title** (Arabic and English), reflecting the suite's Arabic-first design.
- The **question** itself — required, because a consultation without a question is not a consultation.
- A **type** and a **priority**, each defaulting sensibly (**general** and **medium** respectively) when not supplied.
- The **requester** — who is asking, captured both as an identity reference where available and as a name, plus an optional department.
- An optional **link to a legal request**, which back-links the consultation to the originating service-desk request when it was spawned from the catalogue, or is left empty when the consultation stands alone.
- Optional **tags** and structured **metadata**.

The bilingual title and the flexible requester capture make the module equally usable for a formal, catalogue-driven opinion request and for an ad-hoc question raised on someone's behalf. The optional request link is what allows advisory work spawned from the self-service portal to remain joined up with its origin, so the business can always trace an answer back to the request that prompted it.

5.4 Routing to Counsel

Triaged consultations are **routed** to an advisor. Routing records which advisor is responsible (and, optionally, their name for display), together with any routing notes, and advances the consultation from **classified** to **routed**. From this point the assigned advisor owns the answer.

Routing is the mechanism by which advisory demand is matched to advisory capacity and expertise. Because consultations are classified by subject matter before they are routed, the organisation can direct labour questions to employment specialists, tax questions to tax counsel, and so on. The module also surfaces **advisor workload**, so supervisors can see how consultations are distributed across the team and route new work to where there is capacity rather than piling it onto whoever is most visible.

5.5 Advisory Response and Assisted Drafting

The heart of a consultation is the **response** — the advice itself. Recording a response advances the consultation from **routed** to **responded** and captures who responded and when, alongside the body of the advice, any notes, and metadata.

The module supports **assisted drafting** of the advisory response. Two modes are available:

- **Draft-for-preview.** An advisor can request an assisted-drafted first-response memo body **without** transitioning the consultation, purely to preview a starting point. The drafting language can be hinted (defaulting to Arabic), and the advisor remains free to accept, edit, or discard the suggestion. This lets counsel begin from a structured draft rather than a blank page, while keeping the human firmly in control of the answer.
- **Assisted response on record.** When recording the response for real, the advisor can request an assisted-drafted body if no explicit text is supplied, again with a language hint.

In both modes the assisted draft is a productivity aid, not an autonomous answer: the response only becomes part of the record when the advisor commits it, and the sign-off step (below) ensures a second human reviews it. This is the appropriate posture for legal advice — speed on the first draft, human judgement on what is actually relied upon.

5.6 Sign-Off and Separation of Duties

A recorded response is not automatically an approved one. Consultation answers can be put through a **sign-off chain** built on the same shared approval engine used across the suite. The consultation-answer decision is a deliberate **separation-of-duties control point**: the advisor who **authored** the response cannot also be the one who signs it off. This mirrors the pattern applied to investigations and cases and is enforced by the same author-versus-actor parity check, so advisory answers of consequence receive a genuine second-pair-of-eyes review before they are treated as final.

Outstanding sign-off tasks for a consultation can be listed, and the decision is recorded against the approval task. On approval the consultation advances to **approved**, ready to be archived into the knowledge base.

5.7 Turnaround SLAs

Responsiveness is the single attribute by which the business most often judges its legal advisory function, so consultations carry their own working-day **service-level clock**. The clock materialises two windows — an **acknowledgement** window and a **response (turnaround)** window — differentiated by urgency:

Urgency tier	Acknowledgement window	Response (turnaround) window
Urgent	4 working hours	3 working days
Normal	1 working day	5 working days

These are the out-of-the-box defaults; an organisation's administrators can override them per tenant, and the constructor always installs a working default so that a clock always exists. The consultation priority maps onto the two-tier urgency, and the windows are computed in working

time against the organisation's frozen working calendar — Sunday-to-Thursday, honouring official holidays — so the targets reflect real business days rather than raw elapsed time.

When a consultation is spawned from, or otherwise linked to, a legal request, the module also **nudges the linked request's shared SLA clock**, so that the request-side and consultation-side timing stay aligned rather than diverging. This bridging is idempotent and best-effort: it keeps the two views consistent without ever letting an SLA-bookkeeping hiccup block the substantive advisory work.

Because both the consultation clock and the request clock are computed on the same working-calendar foundation used everywhere in the suite, turnaround performance on advisory work rolls up into the same KPI and reporting fabric as the rest of the legal department's SLAs — giving management a single, consistent picture of responsiveness across contracts, requests, and consultations alike.

5.8 Knowledge Reuse and the Advisory Knowledge Base

A question answered once should not have to be answered from scratch again. The consultation lifecycle culminates in **archiving**: an approved consultation is filed away with an optional reason, moving it to the **archived** state. Archived consultations are not discarded — they become the organisation's **advisory knowledge base**.

Several design choices make this knowledge genuinely reusable:

- **Subject-matter classification** organises the archive along the same axes counsel actually think in — contractual, labor, regulatory, corporate, tax, and so on — so a previously answered question can be found by topic.
- **Tags** provide a finer, organisation-defined layer of retrieval on top of the type.
- **Bilingual titles** make the archive searchable and browsable in both Arabic and English.
- **Full attribution and history** mean that a reused answer carries its provenance: who asked, who answered, who approved, and when.

The practical payoff is compounding leverage. As the archive grows, more incoming questions can be answered by pointing to, or adapting, an existing approved consultation, which shortens turnaround, improves consistency of advice across the organisation, and frees senior counsel to focus on the genuinely novel questions.

5.9 Supporting Capabilities

Around the core lifecycle, the module provides the operational tooling a busy advisory practice needs:

- **Document attachments.** Files held in the platform's secure file service can be linked to a consultation — background documents, supporting exhibits, or the drafted memo — with each file linked at most once per consultation, and with file metadata (name, size, content type, kind) recorded against the link.
- **Bulk operations.** Supervisors can act across many consultations at once — archiving, deleting, classifying, routing, or tagging in bulk. Each consultation in a bulk action is processed in its own transaction, so one failure never rolls back the rest; the operation returns a clear per-item result

listing which succeeded and which failed and why. Bulk deletion, being destructive, additionally requires the elevated close-class permission on top of ordinary write rights.

- **Analytics and workload views.** The module exposes counts, statistics, tag lists, and advisor-workload views, giving supervisors the situational awareness to balance the team's advisory load and track advisory throughput.
- **Legal-hold awareness.** Consultations respect the platform's legal-hold controls, so advisory records connected to a matter under a preservation obligation are protected from alteration in the same way as the rest of the suite's records.
- **Immutable audit trail.** As with investigations, every lifecycle mutation on a consultation is written to an append-only governance audit log that admits inserts but not updates or deletions, and the whole module is protected by strict, database-enforced tenant isolation.

Taken together, these capabilities turn legal advisory from an invisible, unmeasured activity into a governed, measured, and reusable service — one whose responsiveness the business can see and whose accumulated wisdom the organisation can bank.

6. Legal Requests & Service Catalog

6.1 Overview and Business Purpose

For most people in an organisation, the legal department is a service they need to call upon rather than a discipline they practise. They have a contract that needs reviewing, a supplier dispute that may need litigation support, a power of attorney to prepare, or simply a legal question they do not know how to categorise. Historically, these needs reach legal through a patchwork of emails, phone calls, and personal favours — a process that is invisible to management, impossible to measure, and unfair to whoever happens to be least able to say no.

The Watheeq **Legal Requests & Service Catalog** module replaces that patchwork with a proper **self-service legal service desk**. It publishes a catalogue of the legal services the department offers, gives the whole organisation a single, structured way to request them, and then drives each request through a governed lifecycle: intake, approval, routing, execution, and delivery — all tracked, timed against service-level targets, and audited. Crucially, it is also the module that **converts a request into work**: when a request is routed, the system automatically spawns the appropriate downstream record — a litigation case or a consultation — so the service desk and the legal team's substantive workspaces stay joined up end to end.

The **legal request** is the canonical spine of this whole flow. Every service-desk interaction, and every downstream domain that arises from it, references the request. The spine is deliberately decoupled from the catalogue and the organisational registry, so that the request machinery works robustly even where those references are optional — but in the configured Watheeq deployment they come together into a single, coherent self-service experience.

6.2 The Service Catalogue

At the front of the service desk is the **service catalogue** — the published, administrator-editable menu of legal services the department offers. Each catalogue entry carries a unique code, a request type, a bilingual name and description, its availability, its approval requirements, its intake channel, and an active flag. Because names and descriptions are bilingual (Arabic and English), the catalogue presents naturally to an Arabic-first workforce while remaining fully usable in English.

6.2.1 The Eight Seeded Services

Every organisation is provisioned, by default, with a standard catalogue of **eight legal services**, so the service desk is useful from day one rather than requiring configuration before anyone can use it. The seeded services are:

#	Service (English)	Service (Arabic)	Requester approval	Provider approval	Channel
1	Legal Consultation — request a general legal consultation	استشارة قانونية	No	Yes	Platform & Email
2	Contract Review —	مراجعة عقد	Yes	Yes	Platform & Email

	review and drafting of contracts and agreements				
3	Contract Drafting — prepare a new contract draft	صياغة عقد	Yes	Yes	Platform & Email
4	Litigation Support — support for cases and judicial disputes	دعم التقاضي	Yes	Yes	Platform & Email
5	Legal Opinion — request a formal legal opinion	رأي قانوني	No	Yes	Platform & Email
6	Regulatory Compliance — review compliance with regulations	الامتثال التنظيمي	No	Yes	Platform & Email
7	Power of Attorney — prepare or review a power of attorney	توكيل	Yes	Yes	Platform & Email
8	General Legal Request — an uncategorised legal service request	طلب قانوني عام	No	No	Platform & Email

The seeding is applied per organisation and is idempotent — services are keyed by a unique code per tenant, so re-running the provisioning never produces duplicates. Administrators can then extend, amend, activate, or deactivate services to fit the organisation's own service portfolio. The presence of a catch-all **General Legal Request** means there is always a valid front door even for needs that do not fit a defined service.

6.2.2 Channels, Approval Requirements, and Eligibility

Each catalogue entry configures several behaviours that shape how a request for that service is handled:

- **Channel.** A service can be offered on the **platform** (self-service portal), by **email**, or **both**. All eight seeded services are available through both channels, so requesters can raise them either through the portal or by emailing a dedicated intake address.
- **Approval requirements.** A service declares whether it requires **requester-side approval** (a sign-off from the requesting side before the work proceeds) and/or **provider-side approval** (a sign-off from the legal department). These flags drive the request's approval routing. For example, **Contract Review** requires both, while a **General Legal Request** requires neither and can flow straight through.
- **Eligibility rules.** A service can carry eligibility predicates that determine who is permitted to request it. Eligibility is evaluated as an **any-of** rule set: a requester is eligible if at least one rule

matches, and a service with no rules is open to everyone. The catalogue exposes an eligibility-check so the portal can tell a requester up front whether they may raise a given service, rather than letting them fill in a form only to be refused later.

6.3 Request Intake

A requester raises a request either through the **self-service portal** or by **email**, and in both cases the outcome is a structured request on the spine.

6.3.1 Structured Intake (Portal)

The portal intake form captures a rich, structured payload:

- A **request type** and an optional link to the chosen **catalogue service**.
- A **bilingual title** (Arabic-first) and a **description**.
- The **requester** — captured both as an identity reference where available and as a name — plus an optional **department** and an optional **beneficiary entity** (the organisational unit on whose behalf the request is made).
- A **priority** — either **normal** or **urgent** — defaulting to **normal**.
- Approval-requirement flags mirroring the service's configuration (requester approval required, provider approval required).
- An optional **subject** reference (type and id), for tying the request to an existing domain object, plus structured **metadata**.
- A **request number**, unique per organisation among live requests, supplied or system-generated.

A newly created request begins life as a **draft**, giving the requester the chance to complete and review it before submitting.

6.3.2 Guarded Urgency

Because "urgent" is a scarce resource that loses meaning if everyone claims it, the module actively guards the urgent priority. A request may only be marked **urgent** if it carries a substantive urgency justification — the justification must be present, of meaningful length, and it must not consist merely of excuses about the requester's own delay or poor planning. The system rejects justifications that amount to "I forgot", "last minute", "poor planning", "ran out of time", and similar phrasings — and it does so in both **Arabic and English**, so the guard cannot be side-stepped by writing the excuse in either language. This ensures that urgency reflects genuine business exigency, not a requester's failure to plan, and it protects the legal team's capacity for the requests that truly need expediting.

6.3.3 Email Intake

For the email channel, the module runs a governed email-intake pipeline. Dedicated intake mailboxes are configured per service, each protected by a signing secret. Inbound email is verified against the addressed mailbox's secret, de-duplicated by message identifier so the same email is never ingested twice, and its body and attachments are preserved. Valid, verified messages are turned into requests on the same spine as portal submissions, so email-originated and portal-originated requests are handled identically from that point on. This lets the organisation offer a familiar "just email legal" front door while still capturing every such request as a governed, tracked item.

6.4 The Request Lifecycle

Every request travels a controlled lifecycle on the spine. The states are:

State	Meaning
Draft	Created but not yet submitted; the requester can still edit it.
Submitted	Handed to legal; awaiting approval routing.
Pending Requester Approval	Awaiting a sign-off from the requesting side.
Pending Provider Approval	Awaiting a sign-off from the legal department.
Approved	All required approvals obtained; ready to be routed.
Routed	Assigned into execution and back-linked to the spawned downstream work.
In Execution	The legal team is actively performing the requested work.
Delivered	The work has been completed and delivered.
Closed	The request is concluded.
Returned	Sent back to the requester for correction/completion.
Cancelled	Abandoned before completion.

The permitted transitions between these states are enforced by the system, so a request cannot skip a required approval or jump straight from draft to delivered. A **returned** request can be corrected and re-submitted; a request can be **cancelled** from most active states; and the approval-bearing states exist precisely to accommodate the requester-side and provider-side sign-offs that the catalogue service declared necessary.

6.4.1 Submission and Approval Routing

Submitting a **draft** (or a **returned**) request moves it forward and engages whatever approvals its service configuration requires. Where a service requires no approvals at all, the module can carry the request straight through: after a no-approval submission it will **auto-route** the request immediately, so a service that needs no sign-off does not sit idle waiting for a manual push. Where approvals are required, the request pauses in the appropriate pending-approval state until the sign-offs are recorded, and only an **approved** request may be routed onward.

Throughout, priority can only be changed through a dedicated, **audited reclassification** action — not through ordinary edits — and every urgent/normal change is written to an append-only priority-change log. This keeps the record of how and why a request's urgency changed fully transparent and tamper-evident.

6.5 Routing, Assignment, and Conversion into Matters

Routing is the pivotal transition where a request stops being a service-desk ticket and becomes actual legal work. When an **approved** request is routed:

8. Its status advances **approved* → routed** under an optimistic-concurrency guard, so two simultaneous route commands cannot both take effect.
9. The system **classifies** the request by its type to decide what downstream work, if any, to spawn.
10. Where a downstream subject is warranted, the system **spawns it automatically** and **back-links** the request to it — recording the subject's type and identifier on the request — atomically, within the same transaction as the status change.

11. The request is now joined up: from the routed request you can reach the case or consultation it produced, and from that downstream record you can trace back to the originating request.

6.5.1 How Requests Convert into Cases and Consultations

The conversion is driven by the request's type, matched case-insensitively and bilingually:

- **Litigation-flavoured requests** — those whose type contains tokens such as *litigation*, *case*, *lawsuit*, or *dispute* (and their Arabic equivalents such as *قضية*, *تقاضي*, *نزاع*) — spawn a **litigation case**. The company is modelled as the plaintiff by default (it filed the request), and the new case begins at its intake status, back-linked to the request.
- **Advisory-flavoured requests** — those whose type contains tokens such as *opinion*, *consultation*, *advice*, *advisory*, *memo* (and Arabic equivalents such as *استشارة*, *رأي*, *فتوى*) — spawn a **consultation**, back-linked to the request, which then follows the advisory lifecycle described in Chapter 5.
- **All other requests** simply route without spawning a downstream subject; they proceed through execution and delivery on the spine itself.

This routing is deliberately robust:

- **Idempotent.** If a request is already routed, re-routing is a harmless no-op that returns the current record; and if a downstream subject already exists for the request — because it was linked on a prior attempt or can be found by the request's identifier — it is reused rather than duplicated. A request never spawns two cases or two consultations.
- **Correlated by events.** When a subject is spawned, the module emits a spine-side correlation event alongside the downstream record's own creation event, so that other systems can join the request to the work it produced.
- **Gracefully degradable.** If the factory for a given downstream type is not wired in a particular deployment, the request still routes cleanly — it simply proceeds without spawning that subject, rather than failing.

The result is a seamless bridge: a business user raises a "litigation support" request through a simple form, and — once it is approved and routed — a fully-formed litigation case appears in the legal team's case workspace, already linked back to the request that asked for it. Likewise a "legal opinion" request becomes a consultation in the advisory queue. The service desk and the substantive legal workspaces are two views of one continuous flow, not two disconnected systems.

6.6 Execution, Delivery, and Tracking

Once routed, a request enters **execution**, where the legal team performs the requested work, and progresses to **delivered** when the work is completed and handed back. The module supports a structured execution phase — capturing completeness requirements, confirming or returning incomplete submissions, and requesting and recording delivery confirmations — so that the handover between requester and legal is explicit and evidenced rather than assumed.

For the requester, the experience is one of **transparency**: from the "My Requests" view they can see each request's current status, its priority, and its history. The request carries a full, append-only **audit trail** of every meaningful action, and its **priority-change history** is separately viewable, so the requester and management alike can see exactly how a request has progressed and how its urgency has been handled.

6.6.1 Service-Level Targets and the Escalation Ladder

Requests are governed by the suite's working-day **service-level machinery**, which is the same fabric used by consultations and investigations, ensuring a consistent measure of responsiveness across the whole legal function. Administrators define SLA targets per service and priority, each target carrying:

- A **turnaround** window in working days.
- An **acknowledgement** window — for urgent requests measured in **working hours** (up to four), and for normal requests in **working days** (up to one).
- A three-rung **escalation ladder** at **2 / 4 / 6 working days**, which is enforced as a fixed standard across targets.

When a request's clock is running, the module materialises the acknowledgement deadline, the turnaround deadline, and the three escalation deadlines, all computed against the organisation's frozen working calendar. If an acknowledgement or turnaround window is missed, the system flags the breach; and as the escalation rungs are reached after a breach, the matter is escalated up the organisational hierarchy — from the **section supervisor** at level one, to the **department manager** at level two, to the **shared-services manager** at level three. The recipients at each rung are resolved from the organisational registry, so escalations reach the right named individuals for the unit that owns the work.

A monitoring process watches the live clocks and drives acknowledgement, breach, and escalation notifications, with de-duplication so that the same rung never fires twice. The live SLA view surfaces, for each request, the working time remaining, the next escalation rung and its recipient, and at-risk / breach-imminent flags — giving supervisors an early-warning picture rather than a post-mortem one. Terminal on-time-versus-breached verdicts feed the quarterly KPI reporting, against which the legal department can demonstrate its adherence to its own service commitments.

6.7 The Requester and Approver Experience

The front-line surfaces bring all of this together for the two audiences who interact with the service desk most:

- **The requester** works from a "My Requests" surface — raising new requests from the catalogue, tracking their status and priority, and following each one from draft through to delivery, with full visibility of its history.
- **The approver and supervisor** work from a cross-domain **Approvals Inbox** — an "awaiting me" work queue that aggregates every decision waiting on the signed-in user across the whole legal suite: service-desk approvals, settlement approvals, governance sign-offs, workflow human tasks, and contract approvals, all normalised into a single list and grouped by kind. Each item shows a live SLA countdown; actionable items offer an inline quick-approve, and the rest deep-

link to the underlying record. A summary strip at the top shows how many decisions are pending, how many are due today, and how many are overdue, so an approver can see their obligations, and their urgency, at a glance.

Both surfaces are bilingual, right-to-left aware, and formatted for the Saudi context, consistent with the Arabic-first design of the whole Watheeq suite.

6.8 Governance, Isolation, and Integrity

As with every module in the suite, the legal-request service desk is built on the same foundations of governance and trust:

- **Strict tenant isolation.** Requests, catalogue entries, eligibility rules, intake mailboxes and messages, priority-change logs, and SLA records are all locked to their owning organisation by database-enforced row-level security. No organisation can see another's requests or catalogue.
- **Append-only audit and priority logs.** Every material action on a request is written to an immutable audit trail, and every priority reclassification to a separate append-only log, neither of which can be altered after the fact.
- **Decoupled but coherent.** The request spine is intentionally decoupled from the catalogue and organisational registry so it is robust and independently deployable, yet in the configured deployment these come together into a single, seamless self-service experience.
- **Authority-gated actions.** Creating, editing, approving, routing, and closing requests, and managing the catalogue, are each governed by distinct permissions, so that sensitive actions — approving a request, managing the published services, deleting a record — are confined to appropriately authorised staff, with destructive actions gated at the elevated close tier.

The outcome is a legal service desk that turns the department's services into a clear, self-service catalogue; captures every need — whether raised through the portal or by email — as a governed, tracked request; enforces sensible discipline around urgency and approvals; measures its own responsiveness against real working-day targets with an escalation ladder that reaches the right managers; and, at the moment of routing, converts each request seamlessly into the case or consultation that fulfils it — closing the loop between the business's request and the legal team's work.

7. Contract Lifecycle Management (CLM)

7.1 Overview

Contract Lifecycle Management is the flagship module of the Watheeq Legal Suite. It carries a contract from the first internal request through drafting, negotiation, formal legal review, multi-tier approval, signature and execution, and finally into a governed, searchable archive — with every state change captured on an immutable audit trail. The design intent is to move the legal department from a filing-cabinet-and-email operating model to a controlled, evidence-producing pipeline where the position of every agreement is always knowable, the responsible owner is always named, and every decision is defensible.

Watheeq treats the contract as a **living record** rather than a static document. Around the core contract entity the suite layers a set of governed sub-systems: a **review desk** that manages document intake and completeness, a **clause and regulation library** that supplies vetted language, a **redline and version-comparison** capability for negotiation, an **approval-routing engine** with delegation-of-authority controls, a **document editor** for collaborative drafting, and an **archive** that preserves the final signed instrument. Each of these is described in this chapter.

Three principles run through the whole module:

- **Every mutation is attributable and auditable.** Status changes, review recommendations, correspondence, attachment re-uploads and archive actions are written to append-only audit tables that carry the acting user, the before/after state and a timestamp. These tables are protected at the database layer so that the record cannot be quietly altered after the fact.
- **Tenant isolation is enforced in the data layer, not just the application.** Every contract table carries the tenant identifier as its first column and is guarded by row-level security policies, so one client organisation can never read or write another's contracts even in the event of an application-layer defect.
- **The lifecycle is a controlled state machine.** A contract can only move between statuses along defined transitions, and the same transition map is enforced on the server and mirrored on the client so the user interface never offers a move the backend will refuse.

7.2 The Contract Record

Every contract in Watheeq is created with a structured header that captures the commercial and legal essentials up front. This structure is what makes the downstream automation — renewal reminders, approval routing, portfolio analytics — possible.

7.2.1 Core Attributes

When a contract is created the following are captured:

Attribute	Purpose
Title and contract number	Human-readable identity and the organisation's own reference number
Type	Classifies the contract (service, supply, NDA, employment, and so on) and drives which approval

	policy and clause set apply
Description	Free-text summary of the deal
Party A / Party B	The two sides of the agreement, each with a name, an optional legal entity, and (for the counterparty) contact details
Total value and currency	The commercial size of the deal; currency defaults to SAR and is normalised to an upper-case ISO code
Payment terms	The agreed commercial payment arrangement
Effective / expiry / renewal dates	The contract's active window and its renewal checkpoint
Auto-renew and renewal notice days	Whether the contract rolls over automatically and how many days of advance notice apply (defaulting to 30)
Owner	The business owner responsible for the contract
Legal reviewer	The assigned legal counsel
Department	The originating business unit
Tags	Free-form labels, de-duplicated and lower-cased for consistent filtering

Sensitive commercial fields are protected: the suite applies **field-level encryption** to designated contract fields at rest, so values such as negotiated terms are stored encrypted and decrypted only when read by an authorised path. Counterparty personal data elsewhere in the suite follows the same pattern.

7.2.2 The Lifecycle State Machine

A contract always occupies exactly one lifecycle status, and it can only move to a status that the transition map permits. The statuses, in lifecycle order, are:

draft → **internal_review** → **legal_review** → **negotiation** → **pending_signature** → **active**, after which a contract can be **suspended**, **renewed**, allowed to **expire**, or **terminated**. **cancelled** is available as an early exit from the pre-signature stages.

The permitted moves are constrained deliberately. For example:

- A **draft** may advance to **internal_review** or be **cancelled** — nothing else.
- **legal_review** may advance to **negotiation**, or step back to **internal_review** or **draft**.
- **pending_signature** may only become **active** or **cancelled**.
- **active** may become **suspended**, **terminated**, **expired**, or **renewed**.
- **terminated**, **renewed** and **cancelled** are terminal — no further transitions are offered.

This map is enforced on the server and is also encoded on the client so that both the contract detail page and the board view only ever attempt moves the backend will accept. The result is that the pipeline cannot be corrupted by an out-of-order status change, whether triggered by a user or by a drag-and-drop on the board.

7.3 Request & Intake

The lifecycle begins before the contract itself exists, at the point a business unit asks the legal department to prepare or review an agreement. Watheeq handles this through a **service-desk request** model that feeds into contract creation, and — once a contract exists — through the **review desk** that governs its formal intake into legal.

The **review desk** is the controlled front door for a contract entering legal review. Opening an intake:

12. Assigns the contract a **desk reference number**, unique within the tenant, so the file can be tracked independently of the contract number.
13. Seeds the four **named attachment slots** the department expects with every submission (see 7.7).
14. Optionally names the **initial legal reviewer** who will own the file.

Intake then progresses through an explicit set of desk statuses — **received** → **acknowledged** → **routed_to_legal** → **under_review** → **returned / completed** — each recorded on the intake record with the acting user and timestamp. A reviewer **acknowledges** receipt, the file is **routed** to the legal desk (optionally re-assigning the reviewer), and if the submission is deficient it can be **returned to the requester** with a mandatory reason and an optional formal deficiency notice, which is surfaced automatically as an entry in the desk correspondence thread. This structured intake replaces the ad-hoc "someone emailed a Word file" model with a numbered, status-tracked, auditable receipt.

7.4 Drafting

WatheeQ provides two complementary drafting paths: **assisted generative drafting** through the AID engine, and **collaborative editing** through an integrated document editor. Both feed into the same review-and-approval pipeline.

7.4.1 The AID Drafting Engine

The suite ships a governed generative-drafting engine — internally the **AID** capability set — that assists lawyers with the mechanical parts of drafting while keeping a human firmly in control. It is bilingual (Arabic/English) and is prompted to draft clear, enforceable, balanced language consistent with **Saudi law and common GCC commercial practice**, to be conservative, and never to invent facts that were not supplied.

The engine exposes a family of drafting actions:

Capability	What it does
Clause generation	Drafts a single contract clause from a plain-language statement of intent, returning the clause text plus a rationale, an assessed risk level, and the assumptions it made
Full-contract auto-draft	Produces a complete contract as ordered sections from a set of deal terms and an optional template hint, including standard protective clauses (governing law, dispute resolution, confidentiality, liability, termination) and a list of open items requiring a human decision
Clause rewrite / re-leveling	Rewrites an existing clause to a target tone and risk posture (favour-us, balanced, favour-counterparty), returning a change log, the direction of the risk shift, and any residual risks
Fallback-clause suggestion	Proposes graduated fallback alternatives for a clause during negotiation, ordered from most favourable to most conceding, each labelled with a concession level and guidance on when to use it
Bilingual translation with equivalence check	Translates legal text between Arabic and English and flags any term whose legal effect does not map cleanly, rating the result as equivalent, partial or divergent

Long-contract key-terms summary	Produces an executive summary and a key-terms brief (obligations, risks, renewal notes) for a long agreement
Defined-term / glossary analysis	Extracts every defined term and its definition, then lists inconsistencies such as undefined-but-used terms or terms used with two different meanings
RFP / tender response drafting	Drafts a structured response to a set of tender requirements, flagging gaps the organisation may not meet
Obligation-extraction QA review	Reviews an automated obligation extraction against the source contract and flags dates, parties or obligations that are not grounded in the text

Critically, the drafting engine is **governed**. Every generation is tenant-scoped, runs against the client's own configured AI credentials, and is routed through the platform's AI-governance prediction logger so the call is **audited** under its own model identity. Unlike the analysis features, drafting deliberately has **no deterministic fallback** — you cannot mechanically manufacture a clause — so when AI governance is fully configured the call is fully governed, and when governance is not configured it degrades gracefully to a still per-tenant-credentialled call rather than failing. The engine also applies hard cost-control bounds (input-length truncation and a token ceiling) so a runaway prompt cannot generate an unbounded bill.

7.4.2 AI Drafts Enter the Human Workflow

An AI-generated draft is never treated as final. Any drafting output can be **submitted for review as a first-class human task** in the shared workflow engine: the draft content is carried into the task's form data, an assignee or assignee-role and an optional SLA are set, and the reviewer sees the draft as the authoritative source of truth to accept or revise. This closes the loop between generative assistance and human accountability — the AI proposes, a named lawyer disposes, and the decision is recorded.

7.4.3 The Collaborative Document Editor

For documents that need genuine multi-party editing, Watheeq integrates an **online document editor** (ONLYOFFICE-class) with governance controls suited to legal work:

- **Session modes** — a document can be opened for **view**, **comment**, or **edit**, so an external counsel or a business stakeholder can be given exactly the access they need and no more.
- **Check-out locking** — an editor can acquire an exclusive **checkout lock** with a reason and an expiry, preventing two people from silently overwriting each other's work, with an explicit release step.
- **Preflight gate** — before a document is finalised it can be run through a **preflight** check that returns a set of pass/fail checks with severities and a blocking flag, so a document that fails a mandatory check cannot proceed.
- **Version snapshots** — the editor can be asked to capture a **snapshot** at a point in time, carrying a change summary and version number, producing the version history the redline capability compares.
- **Guest review links** — a time-boxed, scoped review link can be issued to a named external reviewer (with an email, an organisation, an access mode and specific sections), so an outside party can review without an account and without unbounded access.
- **In-editor clause AI actions** — the drafting engine's clause actions can be invoked directly on a selection inside the editor.

- **Privileged controls** — sensitive editor controls are gated behind a request-with-reason step so they leave an audit trail.

7.5 Clause Library & Templates

Good drafting starts from good building blocks. Watheeq maintains a governed **clause library** and a parallel **regulation library**, both bilingual and both subject to a governance workflow so that only vetted language reaches a live contract.

7.5.1 The Clause Library

Each clause-library item carries a code, bilingual title and text (English and Arabic), a **clause type**, a category, a jurisdiction (defaulting to Saudi Arabia), and provenance (source and source URL). Clauses are **versioned** — a new version can explicitly **supersede** an older one, and an item can be marked **deprecated by** a replacement — so the library keeps a clean lineage of how standard language has evolved rather than silently mutating in place. Tags support fast retrieval.

Every clause carries two independent states:

- A **lifecycle status** (for example draft or active), and
- A **governance status** that starts at **pending review**.

A clause only becomes usable once it clears its **governance decision** — a recorded approve/reject with notes and supporting evidence, after which it can be activated. This means the phrase a lawyer pulls from the library is not merely "a clause someone once wrote" but a clause that has been formally reviewed and approved for reuse, with the approval on record.

7.5.2 The Regulation Library and Clause ↔ Regulation Links

Alongside clauses, the suite maintains a **regulation library** of the statutes, regulations and authority guidance relevant to the jurisdiction, each with bilingual titles and descriptions, an issuing authority, a regulation type, an effective date and provenance. Clauses can be **linked to regulations** with a reference type (for example, a clause that **implements** a particular regulatory requirement). This linkage is what lets the compliance features answer "which of our standard clauses satisfy this regulation, and where is that clause used?" — turning the library from a text bank into a compliance map.

7.5.3 Clause Playbooks

The suite also supports **clause playbooks** — curated positions and fallback ladders for a clause type — which underpin the AID fallback-suggestion capability and give negotiators a house standard, a preferred position, and a set of pre-approved concessions to fall back on.

7.6 Redline & Version Comparison

Negotiation is where contracts change hands and change words, and Watheeq treats the **redline** as a first-class artefact rather than an emailed comparison.

7.6.1 Track-Changes Redlining

The suite computes a **redline** — a track-changes diff — between any two versions of a text. The comparison produces a flat, source-ordered list of segments, each classified as **equal** (unchanged in both), **added** (an insertion in the revised text), or **removed** (a deletion from the original). Because the output preserves source order, a straightforward replacement surfaces as a **removed** segment immediately followed by an **added** one, which the interface pairs into the familiar inline strikethrough-plus-insertion presentation lawyers expect.

The redline can be computed at three levels of granularity:

- **Word** level (the default) — the most readable for prose contracts,
- **Line** level — useful for structured or list-heavy documents, and
- **Character** level — for fine-grained inspection of a small change.

This lets a reviewer look at a negotiation exchange the way they would in a word processor's track-changes mode, but inside the governed system where the versions and the review sit on the same record.

7.6.2 Version Lineage

Redlining is meaningful because the platform keeps a genuine **version history**. The document editor produces version **snapshots** on demand, each carrying a change summary, so the redline always compares two real, captured states of the document rather than two loose files. Combined with the clause-level review and amendment tracking described below, this gives a complete answer to "what changed, who changed it, and why" across the life of the agreement.

7.6.3 Clause-Level Collaboration

Below the whole-document redline, Watheeq supports **clause-level** collaboration on a contract: per-clause **comments** (threaded discussion attached to a specific clause), per-clause **amendments** (proposed changes tracked against the clause), and a **clause review panel**. There is also a **deviation triage** mechanism: where a contract's clauses deviate from the house standard for their clause type, a reviewer can record a disposition — **accept** the deviation, request a **fix**, **reject** it, or leave it **open** — one disposition per clause type, upserted in place, so the team can systematically clear a contract's deviations before approval.

7.7 The Review Desk

The **review desk** is the operational heart of legal review. It is a governed workspace that sits on top of the contract and coordinates the document intake, the completeness check, the reviewer's correspondence, and the final recommendation. It was designed to extend the contract module without disturbing it — every desk table hangs off the contract by reference and enforces tenant isolation.

7.7.1 Named Attachment Slots and Completeness

A submission is expected to arrive with a defined set of documents. The desk seeds **four named attachment slots**:

Slot	Typical content
Draft	The draft contract itself
Quotation	The commercial quotation supporting the deal
Commercial registration	The counterparty's commercial-registration certificate
Committee decision	The internal committee/authorisation decision

Each slot can be toggled **required** or optional and re-labelled, and each holds exactly **one live attachment** — a re-upload into an occupied slot **supersedes** the previous document (the prior version is retained but marked superseded), giving a clean current-state view while preserving history. The desk continuously evaluates **completeness** against the required slots, and this completeness result gates progression: a file that is missing a required document can be **returned** rather than allowed to consume reviewer time.

7.7.2 Correspondence Thread

The desk maintains an **append-only correspondence thread** on the contract. Entries are classified as **internal** notes, **clarification** exchanges, or **auto-return** notices (generated when a file is returned for a deficiency). Because the thread is append-only at the database level — writes are permitted but updates and deletes are not — it forms a tamper-evident record of the back-and-forth around the review.

7.7.3 The Final Recommendation

Review concludes with a recorded **recommendation**, of which only one is live per contract at a time. The outcome is one of:

- **Approved** — optionally handing the file straight to the approval engine (see 7.8) for the Contracts-Manager sign-off,
- **Needs amendment** — requiring a mandatory amendment reason, or
- **Rejected** — requiring a mandatory rejection reason.

The recommendation carries a summary and the recommender's identity, and the whole desk lifecycle — intake status changes, completeness checks, recommendations — is written to an **immutable desk audit trail**. This is what lets the organisation demonstrate, after the fact, that a contract received a proper legal review, on what documents, with what conclusion, and by whom.

7.8 Approval Routing

Once a contract is ready for sign-off, Watheeq routes it through a governed **approval engine** built on the platform's shared workflow engine. This is not a simple "click approve" — it is a policy-driven routing layer with genuine authority controls.

7.8.1 Approval Policies

An **approval policy** describes who must approve a contract and under what conditions. Policies are matched to a contract by **type**, **department**, and **value band** (minimum/maximum value in a given currency), and are ordered by **priority** so the most specific applicable policy wins. A policy defines:

- Its **approvers** — resolved by role or by specific user,
- A **mode** and **quorum** (for example, all approvers, any one, or *N* of them),

- Optional **form fields** the approver must complete, which can themselves be **conditionally visible** based on an expression, and
- An **effective window** (valid-from / valid-until) so a policy can be time-boxed and cleanly retired.

Policies can be created from **templates** and are versioned and audited, so the routing rules themselves are governed artefacts rather than tribal knowledge.

7.8.2 Delegation of Authority (DoA)

A distinctive control in Watheeq's approval engine is **cryptographic delegation-of-authority evidence**. Where a policy requires proof that the approver actually holds the financial authority they are exercising, the approver can supply a signed authority credential — a certificate, a detached signature and a signed payload — which the engine validates against the configured trusted roots (checking the certificate chain, the signature, and the validity window) and confirms that the cryptographically-bound authority amount meets or exceeds the amount the policy requires. Plain-text evidence continues to work where no trusted roots are configured, but where they are, this turns "I'm authorised to sign for this amount" from an assertion into a verifiable, cryptographically-bound fact.

7.8.3 Out-of-Office Delegation, Bulk Decisions and Conditional Forms

The engine also supports operational realities of an approval chain:

- **Out-of-office delegation** — an approver can delegate their pending decisions to a colleague for a defined window, with a reason and evidence, so approvals do not stall when someone is away.
- **Bulk decisions** — an approver facing many similar items can decide them together in one governed action.
- **Conditional approval forms** — approver form fields can appear or hide based on an expression evaluated against the submission, keeping the approver's screen relevant.

Each decision — approve, reject, or delegate — is captured with the actor, the notes, any submitted form data and any authority evidence, feeding the audit trail.

7.9 Execution

After approval, the contract moves into **execution**: gathering the last required items, confirming completeness, capturing signature, and confirming delivery.

7.9.1 Requirement Checklist and the Execution Clock

Execution is governed by a **requirement checklist** — the set of attachments and data items the request or contract needs before work can be considered delivered. Items are typically seeded from the service-catalog configuration and can be edited manually. An item is **satisfied** by attaching the required file or supplying the required data value. When all required items are satisfied, the provider **confirms completeness**, and this action **starts the execution clock** — the point from which the service-level agreement is measured. The confirming party can select which **working calendar** governs the SLA maths (so weekends and Saudi public holidays are handled correctly) and can pin the SLA target.

If a submission is found incomplete, it can be **returned as incomplete**, opening a review round; after a second return the engine auto-closes and clones the request rather than looping indefinitely.

7.9.2 Signature Capture

Watheeq records the **signed date** on the contract and supports a signature sub-system for capturing and tracking signatures, so the transition into **active** status is backed by a recorded execution event rather than an assumption.

7.9.3 Delivery Confirmation

For work that is delivered to a requester, the suite runs a **delivery-confirmation handshake**: after delivery the recipient is asked to confirm or deny receipt, and a working-hours auto-close deadline is computed from the working calendar so an unanswered confirmation does not hang forever. The confirm/deny response is recorded, closing the loop on delivery.

7.10 The Contract Repository & Archive

A contract's life does not end at signature — it ends (for record-keeping purposes) in the **archive**, and Watheeq treats archiving as a deliberate, governed action.

7.10.1 Archiving

Archiving is a **soft state** distinct from the contract's business status: a contract stays in its current lifecycle status (for example, **expired** or **terminated**) while its **archive status** flips from **active** to **archived**. Archiving records the **archive date**, the **user who archived it**, and an **archive reason**, so the act of putting a contract to rest is itself attributable. An indexed **Archived Contracts** view lets the team browse and filter the archive by archive status and date efficiently, with a dedicated filter rail in the interface.

7.10.2 The Final Version and Document Manifest

The archive is anchored on the **final executed version** of the contract. The suite records when the final version was uploaded and maintains a **document archive manifest**, so the authoritative signed instrument is preserved and identifiable — the record you would produce in a dispute or an audit. Combined with the immutable audit trails from intake, review, approval and execution, the archived contract is a complete, defensible package: the agreement, its history, and the evidence of how it was reviewed and approved.

7.11 The CLM Frontend

The CLM user experience is built to serve real legal-operations work: fast triage of a large portfolio, deep work on a single contract, and negotiation-grade document handling.

7.11.1 List, Board and Calendar Views

The contracts surface offers three interchangeable views, and the user's choice is **persisted** across sessions (stored locally, kept in sync across browser tabs):

- **Table view** — the dense, filterable list, with adjustable **row density** (comfortable or compact) and **hideable columns**, for portfolio triage.

- **Board view** — a lifecycle **Kanban** with a column per status in lifecycle order, colour-coded by semantic tone (in-flight review states in sky, time-pressured states such as negotiation and pending-signature in amber, healthy active/renewed states in emerald, terminal states in rose/neutral). Crucially, a drag-to-move on the board only attempts transitions the backend will accept, because the board shares the same transition map as the detail page.
- **Calendar view** — surfaces the time dimension of the portfolio (effective, expiry and renewal dates) so upcoming key dates are visible at a glance.

7.11.2 Saved Views and Presets

The interface supports **saved views** and preset filters so a user can capture a working set — for example "my department's contracts up for renewal in 90 days" — and return to it directly, mirroring the saved-view idiom used across the suite. **Renewal warnings** are surfaced proactively as banners so nothing lapses unnoticed.

7.11.3 The Contract Detail Workspace

A single contract opens into a rich workspace with a **lifecycle stepper** showing where the contract sits, a **key-dates strip**, **risk findings** and a **risk panel**, a **clauses tab** with per-clause comments, amendments and a review panel, a **compliance tab**, a **categorisation** helper, and the **review-desk** components including the **final-version** capture. This is where the redline, the document editor, the clause library and the approval engine all come together on one screen, so a lawyer can do the whole job of reviewing, negotiating, and progressing a contract without leaving the record.

7.11.4 Document Viewer and Redline UI

The redline is rendered through a shared **redline view** that consumes the segment model described in 7.6, presenting insertions and deletions inline. The document editor is embedded for collaborative drafting with the session modes, locking, snapshots and guest-review links already described. Together these give the negotiation experience the fidelity of a desktop word processor while keeping the work inside the governed, audited platform.

8. Litigation & Hearings Management

8.1 Overview

The litigation module manages the organisation's court disputes end to end, on both sides of the docket: cases the organisation **brings** (plaintiff flows) and cases **brought against** it (defendant flows). It covers pleadings and the statement of claim, hearing scheduling and hearing minutes, court-appointed experts, judgments and the study-and-objection decision, and the deadlines that thread through all of them. As with CLM, the design goal is that no procedural step and no deadline is ever lost, and that every action leaves a defensible, immutable record.

Litigation in Watheeq extends the **legal case** aggregate. Pleadings, expert assignments, judgments and defendant registrations all attach to a legal case; hearing reports attach to a specific hearing on that case. Supporting documents flow through the platform's Files service. Every litigation table enforces tenant isolation and soft-delete, and the module carries a dedicated **append-only litigation audit trail** that records every pleading, judgment and defendant-case transition with its before/after state, the acting user, and the reason — immutable at the database layer, so the governance record cannot be altered after the fact.

A deliberate architectural choice underpins the whole module: **litigation deadlines reuse the existing obligation reminder system rather than inventing a new timer**. When a court deadline needs tracking — an objection deadline, a hearing date, an expert report due date — the module creates a linked **legal obligation**, and the platform's existing obligation reminder outbox and monitor fire the reminders. This means every litigation deadline benefits from the same battle-tested, lead-time-configurable reminder machinery used elsewhere in the suite, with no parallel, untested timer to go wrong.

8.2 Litigation Case Handling

A litigation matter is a **legal case**, and the module distinguishes the two roles the organisation can play.

8.2.1 Plaintiff Cases

When the organisation is the **plaintiff**, the litigation flow centres on preparing and filing **pleadings** — the statement of claim and any subsequent replies and briefs — driving them through internal approval, and filing them with the court. This is described in detail in 8.4 and 8.5.

8.2.2 Defendant Cases

When the organisation is the **defendant**, an incoming lawsuit is formally **registered** as a defendant case against a legal case, capturing:

- The **plaintiff's name**,
- The **court**,
- The **notification date** (when the organisation was served), and
- The **served statement of claim** and supporting documents, uploaded as attachments.

The defendant case then moves through an explicit lifecycle: **registered** → **notified_dept** → **response_drafting** → **response_in_review** → **response_approved** / **response_rejected** → **closed**, with **cancelled** available. Two supporting steps are built in:

- **Najiz representative** — the module records the company's Najiz (ناجز, the Saudi judicial portal) representative and a Najiz status. This is a **manual entry** today, with a real Najiz API integration deferred; the status field distinguishes manual, synced and failed so the data model is ready for automation without pretending it exists.
- **Department notification** — the concerned business department is formally **notified** of the incoming lawsuit, recorded with a timestamp, so the internal stakeholders who own the underlying dispute are looped in on the record.

8.3 Hearing Scheduling & Outcomes

Hearings are managed against the case as scheduled events and are then documented after they occur.

8.3.1 Scheduling

A hearing is recorded against a legal case with a **hearing date**, a **location**, notes and (once known) a **decision**. Hearings are indexed by case and date so the upcoming-hearings picture is always available. Hearing dates can themselves be tracked as **deadline obligations** (see 8.6) so the team is reminded ahead of each session.

8.3.2 Hearing Reports and Minutes

After a hearing, the outcome is captured as a **hearing report** attached to that specific hearing. Reports are typed:

Type	Meaning
Minutes (ضبط الجلسة)	The record/minutes of the session
Decision	A decision issued at the hearing
Report	A general report on the session

Each report carries a title, a body, an optional **decision** text, the **recorded-at** time, and an optional linked file (for example, a scanned court minute). Reports are stored per hearing and per case with full history, giving a complete, chronological account of what happened at each session of the dispute.

8.3.3 Court-Appointed Experts

Where a court appoints an expert (نائب خبير), the module tracks the **expert assignment**: the expert's name and specialisation, contact details, the **mandate**, and the **report due date**, moving through a status lifecycle of **requested** → **appointed** → **report received** → **closed**, with **cancelled** available. Documents furnished to the expert are recorded as **expert documents**. The expert's report due date can be tracked as a deadline so the report is chased on time.

8.4 Statement of Claim & the AID Drafting Engine

Pleadings — the statement of claim and its successors — are the plaintiff-side heart of litigation, and Watheeq brings the same governed generative drafting to them that it brings to contracts.

8.4.1 Pleadings and the Statement of Claim

A **pleading** is created on a case with a type — **statement of claim** (the default), **reply**, **brief**, or **other** — a title and a body. It is assigned a unique **pleading number** within the tenant and moves through a controlled lifecycle: **draft** → **in_approval** → **approved** → **filed**, with **rejected** available. Each pleading tracks a **current version**, and every edit appends an **immutable version snapshot** (insert-only, so drafts cannot be silently rewritten) with a change reason, giving a clean, tamper-evident drafting history. Supporting evidence is attached as **pleading attachments** via the Files service.

8.4.2 AID-Assisted Drafting of the Statement of Claim

The pleading body can be **drafted by the AID engine** rather than written from scratch. When a pleading is created (or a defendant's first-response memo is drafted) with the generate flag set and no body supplied, the service invokes the governed generative-drafting engine — the same bilingual, Saudi/GCC-aware, audited engine described in Chapter 7 — to produce the draft body from a prompt. The pleading is then flagged as **AI-generated**, so the provenance of the draft is on record. As with contract drafting, the AI produces a draft; a human reviews and approves it before it goes anywhere.

8.4.3 Approval and Filing

A drafted pleading is submitted into the shared workflow engine for **approval** — a per-tenant "Litigation Pleading Approval" workflow created lazily on first use — and only an **approved** pleading can be **filed** with the court. Filing records the **filed-at** date and a note, moving the pleading to its terminal **filed** state. On the defendant side, the **first-response memo** (مذكرة الرد الأولى) similarly goes through a **two-tier supervisor** → **section-manager review** before it is approved.

8.5 Judgment Tracking & Idempotency

When the court rules, the judgment is recorded and then **studied** to decide whether to accept it or object — and this is one of the most carefully engineered parts of the module, because a mistake here (a missed or duplicated objection deadline) has real legal consequences.

8.5.1 Recording a Judgment

A **judgment** is recorded on a case with a reference, a date, an **outcome** (won, lost, partial, or other), a summary, and an optional linked file (the judgment document). At this point it is simply on record.

8.5.2 The Study Decision

The judgment is then **studied**: a lawyer records study notes and a **recommendation** — **accept** the judgment, or **object** to it. When the recommendation is **object**, an **objection deadline** is required, and the service creates a linked **legal obligation** so the existing reminder outbox fires the deadline. This is the key design point: the objection deadline is not a new bespoke timer, it is an ordinary obligation, so it inherits the suite's proven reminder behaviour and lead-time configuration.

8.5.3 Idempotency and Concurrency Safety

Because a duplicated objection obligation would produce a reminder storm — and, worse, ambiguity about the real deadline — the study operation is engineered to be **exactly-once**, with defence in depth at three layers:

15. **A row lock and a guard in the service.** The judgment row is locked for update and the operation rejects if the judgment has already been studied, so a second attempt on an already-studied judgment is cleanly refused.
16. **A partial-unique index at the database.** A real judgment-to-obligation link column carries a partial-unique index guaranteeing **at most one live objection obligation per judgment**. Even if two study requests race past the row lock, the second insert violates the index and is refused by the database.
17. **Graceful mapping of the conflict.** The service maps the database uniqueness violation to an idempotent "already studied" conflict, so the caller gets a clean, predictable result rather than an error, and there is never a duplicate reminder.

The result is that no matter how the study operation is called — retried, double-clicked, or hit by two users at once — there is exactly one objection deadline and exactly one reminder chain, and the whole thing is recorded on the immutable litigation audit trail. The same append-only trail records the judgment's creation and study, so the objection decision is fully attributable.

8.6 Deadlines

Deadlines are the connective tissue of litigation, and Watheeq handles them through a single, consistent mechanism: the **deadline obligation**.

Any court date that must be tracked — a hearing, an objection deadline, an expert report due date — can be created as a **deadline obligation** on the matter. The obligation carries:

- A **kind** classifying the deadline (for example, hearing or objection),
- A **due date**,
- A responsible **owner** (defaulting to the caller if not specified), and
- **Reminder lead times** in days, defaulting to seven days and one day before the deadline.

From there the platform's existing **obligation reminder outbox and monitor** dispatch the reminders — no new timer, no parallel scheduler. Because litigation obligations were made valid without a contract or matter link (a case-only link is accepted), a judgment objection deadline can exist purely against the case, exactly as the legal process requires.

Alongside per-deadline reminders, the module tracks **external dependency delays** on a matter — periods where progress is blocked by the **court**, a **government** body, a **department**, or an **expert** — as classified delay windows, and can flag a matter as **externally pending** with a category and reason. This distinguishes "we are late" from "we are waiting on the court", which matters both for SLA fairness and for management reporting on where cases are actually stuck.

9. Settlements & Dispute Resolution

9.1 Overview

Not every dispute goes to judgment; many are resolved by agreement. Watheeq's **settlements and alternative-dispute-resolution (ADR)** module tracks a reconciliation from the first proposal, through negotiation rounds, to approval, execution and closure — with the same discipline the rest of the suite applies: a controlled state machine, an immutable audit trail, encrypted counterparty data, and concurrency-safe closure.

A settlement in Watheeq attaches to a **legal matter**. It records the method of resolution, the negotiated financial terms, the counterparty, and the running negotiation history, and it drives the matter to closure when the settlement is executed. Every settlement carries a unique reference within the tenant, enforces tenant isolation, and supports soft-delete.

9.2 Settlement Tracking

9.2.1 Opening a Reconciliation

A settlement begins when a **reconciliation attempt** is opened on a matter. Opening it captures:

- The **method** — **reconciliation** (the default), **mediation**, **arbitration**, **negotiation**, or **other**,
- A **title** and the proposed **terms**,
- The **value** and **currency** of the proposed settlement, and
- The **counterparty** — name, contact, and identity number.

The settlement starts in status **proposed**. An optional human-readable **reference** can be supplied, otherwise the system assigns one; references are unique per tenant.

9.2.2 The Settlement State Machine

A settlement moves through a defined set of states:

Status	Meaning
proposed	A reconciliation attempt has been opened
negotiating	Negotiation rounds are in progress
pending_approval	Terms are recorded and the settlement is awaiting approval
approved	The settlement has cleared the approval chain
executed	The approved settlement has been executed and has closed the matter
rejected	The approval chain rejected the settlement
abandoned	Reconciliation failed or was withdrawn

The status domain is enforced both in the application and by a database check constraint, so a settlement can never come to rest in an invalid state.

9.2.3 Counterparty Data Protection

Counterparty personal data — name, contact, and identity number — is **field-encrypted at rest**. The service encrypts these values before they are written and decrypts them only on read by an

authorised path, so sensitive personal information about the opposing party is protected in storage, consistent with the suite's data-protection posture and Saudi personal-data expectations.

9.3 Negotiation Records

Settlement is rarely a single number; it is a sequence of offers and counter-offers, and Watheeq preserves that sequence as a first-class record.

Each **negotiation round** is appended to the settlement with:

- A **round number** (sequential and unique within the settlement),
- Who **proposed** it,
- The **proposed value** and currency,
- The **terms** offered, and
- The **outcome** of that round.

Recording a round moves the settlement into the **negotiating** state. The rounds form a complete, ordered ledger of how the parties moved from the opening position to the final agreement — invaluable both for the lawyers running the negotiation and for any later question about how a particular figure was reached. The negotiated terms themselves can be **recorded and updated** on the settlement while it remains mutable, so the settlement's headline terms always reflect the current position while the round-by-round history is preserved underneath.

9.4 Approval

A settlement of any significance requires sign-off, and Watheeq routes it through the same governed approval machinery the rest of the suite uses.

When terms are ready, the settlement is **submitted for approval**, moving it to **pending_approval** and opening a workflow instance in the shared engine. The approval chain then either **approves** the settlement — clearing it for execution — or **rejects** it. Because settlement approval runs on the platform's approval engine, it inherits the same controls described in Chapter 7: policy-driven routing, approver quorums, out-of-office delegation, and — where required — cryptographic delegation-of-authority evidence proving the approver holds the financial authority to bind the organisation to the settlement figure. A settlement that fails approval is **rejected**; one that is never agreed can be **abandoned**.

9.5 Financial Terms

The financial substance of a settlement is captured explicitly and carried through the whole lifecycle:

- The **value** and **currency** of the settlement are recorded on the settlement itself and on each negotiation round, so both the current agreed figure and every intermediate offer are on record.
- The **terms** text captures the qualitative substance — payment schedules, releases, and conditions.
- The **counterparty identity** is captured (and encrypted) so the paying/receiving party is unambiguous.

This structured capture of the financial terms is what lets the settlement feed reporting — for example, aggregate settlement exposure and outcomes — and what makes the executed settlement a complete financial record of how the dispute was resolved.

9.6 Closure

Closing a settlement is the most consequential action in the module, because executing a settlement **closes the underlying matter**, and Watheeq engineers this step to be safe under concurrency.

9.6.1 Close by Reconciliation

When an approved settlement is executed, the **close-by-reconciliation** operation transitions exactly one settlement to **executed** and closes its owning matter, recording the execution time and the closure reason. This is the point at which the dispute is formally resolved on the record.

9.6.2 Double-Close and Concurrency Safety

Because a matter must never be closed twice — and two settlements on the same matter must never both reach executed — closure is protected with defence in depth:

18. **Application-level compare-and-swap.** The service locks the settlement's status for update and applies a compare-and-swap update on both the settlement and the matter, only transitioning from the expected prior state, so a stale or duplicate close is refused in the application.
19. **A partial-unique index at the database.** A partial-unique index guarantees **at most one executed settlement per matter** among live rows. Even if a buggy or racing caller tries to execute a second settlement on the same matter, the database refuses the second execution at commit.

Together these make the invariant — one matter, one executed settlement, one closure — impossible to violate even under concurrent load or a defective caller.

9.6.3 Audit Trail and Documents

Every settlement transition — opening, recording terms, each negotiation round, submission for approval, approval or rejection, and execution — is written to an **append-only settlement audit trail** that captures the action, the from/to status, the acting user and a timestamp, and is immutable at the database layer. Supporting **settlement documents** (the signed reconciliation agreement, releases, proof of payment) are attached to the settlement through the platform's Files service. The result is that a closed settlement is a complete, defensible package: the agreed terms, the full negotiation history, the approval decision, the execution record, the supporting documents, and an immutable trail of every step — everything the organisation would need to demonstrate that the dispute was resolved properly and with authority.

9.7 Where Settlements Fit

Settlements do not stand alone. They are the resolution path that runs in parallel with litigation: a matter being litigated can be settled at any point, and the settlement's execution is what closes the matter. Combined with the litigation module's judgment tracking, this gives the legal department a single, coherent view of a dispute's possible endings — a court judgment (accepted or objected to)

on one path, and a negotiated settlement (proposed, negotiated, approved, executed) on the other — each governed, each audited, and each ending in a clean, attributable closure of the matter.

10. Approvals, Delegation of Authority & Segregation of Duties

10.1 Approvals as the Heart of Legal Governance

In a corporate legal department, the approval is where governance actually happens. It is the point at which authority is exercised, risk is accepted, and accountability is fixed. Watheeq treats approvals accordingly — not as a status flag someone flips, but as a governed decision, taken by the right people, in the right order, to the right quorum, under demonstrable authority, and recorded immutably. This chapter describes the three interlocking mechanisms that make Watheeq's approvals trustworthy: the **approval policy engine** that determines what approval a given piece of work requires; the **delegation of authority validation** that proves the approver had the authority to bind the organization; and the **segregation of duties controls** that stop the same person from playing conflicting parts.

10.2 The Approval Policy Engine

Watheeq does not hardcode who approves what. Instead it carries a first-class **approval policy** — a configurable rule that determines, for a given contract or request, who must approve, in what mode, to what quorum, and under what authority. Policies are the difference between "the system requires an approval" and "the system requires *the right* approval, by *the right* authority, for *this* item."

Scope. An approval policy is scoped by the dimensions that matter to a legal department: for contracts, the contract type, the department, and a value band (a minimum and maximum, with a currency); for requests, the request type, the service, the approval stage (requester side or provider side), the department, and the priority tier. This scoping is what lets a low-value non-disclosure agreement and a high-value acquisition agreement be routed to entirely different approvers automatically.

Routing. Each policy declares its **mode** — sequential or parallel — and its **quorum** — all approvers, any one, or a specified number of a set (an "n of m" quorum). It names its approvers as specific users or as roles, so a policy can say "the contracts section manager then the CFO, in sequence," or "any two of these three directors, in parallel." It can attach **conditional form fields** that an approver must complete, each with a visibility expression so that a field appears only when it is relevant to the decision.

Authority requirement. A policy declares whether **authority evidence** is required and, if so, the required approver role and the required authority amount — the financial limit up to which the approver's authority must extend for the approval to be valid.

Crucially, policies are **governed objects**, not editable settings, and this is what makes them trustworthy in an audit:

- **Versioned.** Every change to a policy snapshots the prior state into an immutable version history under its existing version number, then increments the live version. The organization can

therefore show not just that a contract was approved, but under *which version of which policy* the approval was required and granted. A prior version can be restored — which itself creates a new version and an audit entry, so history is never rewritten, only appended.

- **Audited.** Every create, update, archive, restore, and template application writes an append-only audit entry capturing the before and after state, the actor, and a traceable request identifier, into a table with no update or delete path.
- **Conflict-checked.** Because overlapping policies could contradict each other, the engine computes overlap across all the scope dimensions and **refuses identical scope duplicates outright** (a hard failure on create or update), while surfacing merely overlapping policies as warnings an administrator can resolve deliberately.
- **Effective-windowed.** Policies carry a validity window, so a policy applies only while it is in force, and the recommendation engine never proposes an expired policy for a new approval.
- **Template-backed.** Reusable templates let an organization standardize its approval patterns and instantiate concrete policies from them, with each policy retaining a link to the template it came from.

Recommendation. The engine actively **recommends** the best-matching active, in-window policy for a given contract or request, ranking candidates by priority and by the specificity of their scope, so the correct approval requirement is applied automatically rather than chosen by hand each time — and it reports the match and the reason for transparency.

For a legal director, the payoff is control with auditability: the department's entire authorization matrix lives as versioned, conflict-checked, effective-windowed policy, and every approval can be traced back to the exact rule, at the exact version, that required it.

10.3 Delegation of Authority — Proving the Approver's Authority

An approval is only as good as the authority behind it. Watheeq can require that an approval carry **cryptographic delegation of authority evidence** — turning "an authorized person approved this" into "this specific, cryptographically identified authority, within a validated financial limit, approved this item of this value."

Where a policy requires authority evidence, the approver's decision must present a certificate-backed proof consisting of an X.509 certificate identifying the authority, a detached digital signature over a signed payload, the algorithm used, and (optionally) a trusted-root bundle. The platform validates this in **three prongs**:

20. **Chain validation.** The leaf certificate must chain to a trusted root, be within its validity window, and carry the appropriate digital signature key usage.
21. **Signature verification.** The detached signature must verify over the *exact* signed payload using the certificate's public key. A range of algorithms is supported — ECDSA and RSA, each with SHA-256, SHA-384, or SHA-512.
22. **Amount binding.** The authority amount and currency are parsed from the cryptographically signed payload and checked against both the policy's required authority amount and the value of the item being approved — so an approver whose authority is limited to a certain amount cannot validly approve a commitment above it.

The platform operates this in two modes. In **fallback mode** — where no trusted roots are configured — it accepts plain-text authority evidence and logs a warning, so a department can operate before its public-key infrastructure is provisioned. In **strict mode** — where trusted roots are configured — cryptographic material is mandatory, and the platform returns a **specific, actionable** failure for each failure kind: an expired certificate, an untrusted chain, an invalid signature, an unsupported algorithm, a revoked certificate, or malformed evidence, rather than a generic rejection. Optional offline revocation checking lets an operator mark specific certificate serial numbers revoked. Trust roots, revocation settings, and the certificates themselves are supplied as configuration content, consistent with the platform's key-custody approach.

This is a genuine, and unusual, legal-technology control. It gives the organization the ability to **prove** — not merely assert — the authority under which its most significant legal and financial commitments were approved. In litigation over corporate authority, or in an audit of financial controls, that is a materially stronger position than a name recorded in an approval field.

10.4 Segregation of Duties — Static and Dynamic

The third pillar is segregation of duties, which ensures that no single person can perform conflicting roles in a legal transaction. Watheeq enforces it in two complementary ways.

Static segregation of duties prevents conflicting roles from being assigned to the same person in the first place. The platform defines mutually-exclusive role pairs, seeded per tenant, and checks them at role-assignment time so a user cannot be configured into a self-conflicting position:

- A legal officer (who handles and drafts cases) cannot also be the cases section manager (who approves and closes them).
- A legal advisor (who **recommends** on contracts) cannot also be the contracts section manager (who gives **final sign-off**).
- No operational role may also be the compliance auditor.

Dynamic segregation of duties enforces the *author-is-not-the approver* principle at the moment of decision, on the specific record. Even a user who holds the approval permission is denied the ability to approve or close a record they authored, and a two-round review requires two genuinely distinct approvers — a prior approver cannot approve again at a second tier. This control **fails closed**: if the platform cannot resolve the record's author, or cannot prove the actor is distinct, it denies. And it applies to everyone, with **no administrator bypass** — a privileged operator cannot self-approve.

Reinforcing both is the permission design itself: the elevated verbs — approve, close, assign, distribute, and manage — accept no coarse fallback, so a broad write permission does not silently unlock them. The dynamic check layers **on top** of the permission check, so approving a case requires both the case-approve permission **and** being someone other than the case's author.

The combined effect is that Watheeq structurally enforces the separations a legal department's controls require: the person who drafts is not the person who approves; work allocation (assign, distribute) is a distinct authority from legal sign-off; oversight cannot alter what it oversees; and every one of these boundaries is enforced by the system rather than trusted to individual discipline.

11. Workflow & Automation Engine

11.1 A Real Engine, Not Bespoke Legal Plumbing

Every approval chain, human task, escalation, and multi-step legal process in Watheeq runs on a shared, governed workflow engine — a genuine finite-state-machine engine built into the platform, which the legal suite consumes rather than reimplements. This is a deliberate architectural decision with a direct customer benefit: cases, requests, consultations, investigations, settlements, and contract sign-offs all ride the same execution core, so the reliability and governance invested in that core is inherited uniformly by every legal workflow. It also settles, in the product's favour, the perennial "build or outsource the workflow runtime" question — the engine is a first-party, governed component, not a black-box dependency.

11.2 The Shape of a Workflow

The engine models a workflow as a **definition** — a versioned set of steps and transitions, with typed variables, a trigger configuration, and a category (approval, review, onboarding, escalation, compliance, and so on). It executes **instances** of that definition, each moving through its states: running, completed, failed, cancelled, suspended, and a governed **incident** state. Each step's execution is itself tracked with its own status, and transitions are guarded so that only legal moves occur.

The engine supports a rich vocabulary of step types that together cover the shapes real legal processes take:

Step type	What it does
Human task	Parks the workflow and waits for a person; assignment, work queues, forms, SLA deadlines
Approval chain	Multi-approver decision, sequential or parallel, with quorum and distinct approver enforcement
Service task	Governed call to an internal service, with circuit breaking, idempotency, and bounded retry
Connector task	Governed dispatch to an external integration, with egress controls and secret custody
Decision task	Table-driven (decision-table) routing based on inputs
Condition	Evaluates a boolean guard expression
Parallel gateway	Forks and joins concurrent branches, with all/any/n-of/m completion
Event-based gateway	Races several events (timers or messages) and routes to the first
Timer	Wakes the workflow at a future time (a duration or an absolute instant)
Call activity	Starts a child workflow, maps inputs, and resumes the parent on completion
Multi-instance	Fans work out across many items and aggregates the outputs
End	Terminates the workflow

11.3 Human Tasks — Where People Meet the Process

The human task is the point at which a workflow parks and waits for a person, and it carries the features a real legal process needs: assignment to a specific user or to a role; **candidate?user and candidate?group work queues** so a task can be offered to a pool and claimed; claim, complete, delegate, and reject semantics; **out?of?office substitution** so that when an assignee is away, the task re?routes to their deputy along a resolved substitution chain; SLA deadlines with breach detection and escalation; a priority; and **dynamic forms** whose fields can be conditionally shown and whose sensitive fields can be classified for encryption. This is the machinery behind every legal approval, review, and decision that a person must make.

11.4 Governed Reliability

The engine is built for governed reliability, which matters enormously for legal work where a lost or double?executed approval is a real harm:

- **Atomic transitions.** A step transition and its audit record commit in a single transaction, so the state and its evidence never diverge.
- **Serialized instances.** Instances are serialized with advisory locks and optimistic concurrency, so a workflow cannot double?advance under concurrent events.
- **Governed incidents.** When a step exhausts its retries, the engine does not kill the whole process. It parks the failed step as a **governed incident** — following the maker?checker incident pattern — with an override flow and a dead?letter path, so a stuck legal approval becomes a managed incident an administrator can retry, skip, or override, rather than a silently lost workflow.
- **Encrypted payloads and events.** Sensitive workflow payloads can be encrypted, and every material event is emitted for downstream subscribers.

11.5 Standards, Expressions, Promotion

The engine speaks the industry?standard **BPMN 2.0** notation for import and export, so processes can be visualized in standard tools and exchanged, with a fail?closed importer that reports unsupported constructs rather than silently dropping them. Conditions and variable references are evaluated by a safe, sandboxed expression evaluator with a sanitizer, so process logic can be expressive without being a security risk. And definitions move through a **promotion model** — development, then staging, then production — with immutability set once a version reaches staging or production and an optional production?approval gate that requires a distinct approver to promote to production. A change to a legal process is therefore itself a governed, staged, approvable act.

11.6 How Watheeq Consumes the Engine

Watheeq wraps the engine in a **subject?agnostic approval orchestrator** and a set of per?domain orchestrators — for cases, investigations, requests, settlements, drafting reviews, and litigation steps. A specification object describes each subject (its table, its identifier, how to hook its status, how to emit its events), which lets the *same* orchestrator drive a case?intake directive chain, a contract sign?off, a settlement approval, and a consultation response without re?implementing the mechanics each time. The orchestrator mirrors the engine's transactional decision semantics: it locks the record and the task, validates the actor, validates the form, validates the delegation?of?authority evidence, resolves the quorum, advances the state machine, and emits a

domain event — all in one governed transaction. The drafting workflow illustrates the integration: a generated draft is delivered into the engine as a tracked human review task carrying the draft content, so human review of a draft is a first-class, audited step in the same engine that governs every other approval.

11.7 What This Means for the Business

For a legal department, the engine turns *process* into governed *infrastructure*. Legal processes are modelled explicitly, executed reliably, escalated automatically, recovered gracefully when a step fails, versioned and staged when they change, and audited throughout. And because it is shared infrastructure, the same guarantees apply whether the process is a two-signature contract approval or a multi-phase litigation intake with cryptographic authority evidence. The department is not maintaining a pile of ad-hoc scripts; it is running its legal operations on a governed process engine.

12. AI-Assisted Legal

12.1 AI Where It Helps, Governed Where It Matters

WatheeQ applies artificial intelligence to the legal tasks where it genuinely accelerates a lawyer — reading and analyzing contracts, and drafting legal text — while wrapping that AI in governance appropriate to a legal system of record. The design principle is that AI **assists** the lawyer and is **always subordinate to human review and to governed controls**; it never silently makes a legal decision, and the system degrades gracefully to deterministic behaviour when AI is unavailable or when a governed prediction is rejected. This is the posture a legal department needs: real acceleration, without ceding legal judgment or accountability to a model.

12.2 Contract Analysis — Clause and Obligation Extraction

When a contract is reviewed, WatheeQ can use a governed large-language-model enrichment to **extract clauses and obligations** — surfacing the terms and duties a reviewer must consider so that nothing material is silently missed. This is deliberately built as an **enrichment over a deterministic analyzer**: if the AI is unavailable, or if the governance layer rejects the prediction, the platform falls back to the deterministic analysis rather than failing or fabricating. The reviewer always sees a result; the AI makes that result richer when it is available and trustworthy. The extractors are registered, governed model functions, so their use is tracked and auditable rather than an opaque call to an external service.

12.3 AI-Assisted Drafting

WatheeQ's drafting engine generates legal text — assisting with the production of drafts such as statements of claim and other legal instruments — through a governed model under each tenant's governance context. Critically, a generated draft does not bypass review: it is delivered into the workflow engine as a **tracked human-review task carrying the draft content**, so a lawyer reviews, edits, and approves the draft through the same governed, audited process that applies to any other legal work product. AI accelerates the first draft; the lawyer remains accountable for the final instrument, and the review is a first-class, recorded step rather than an informal side conversation.

12.4 AI Governance, Metering, and Guardrails

Because AI in a legal context must be accountable and cost-controlled, WatheeQ runs it under real governance rather than as an unbounded API call:

- **Per-tenant governance and audit.** AI predictions are tenant-scoped and logged, with the input (or its hash), the confidence, the explanation, and the model and version recorded — an audit trail for the AI's **contribution**, not merely its output. This means the organization can answer, after the fact, what the AI was asked and what it produced.
- **Metering and cost control.** Language-model usage is metered per tenant — token consumption and estimated cost are tracked and aggregated into daily and monthly rollups — and hard guardrails cap usage: per-tenant limits on calls per minute, per hour, and per day, on tokens per day, and, importantly, on **cost per day**. When a tenant reaches its daily cost ceiling, further AI

calls are refused rather than silently run up a bill. The organization's exposure to AI cost is therefore bounded by policy, not by trust.

- **Provider flexibility and sovereignty.** The platform can use different AI providers, including **self-hosted local models** for sovereign or air-gapped deployments where sending legal text to an external AI service is unacceptable. In those deployments, AI assistance runs entirely within the customer's own environment at a fixed infrastructure cost, with no legal content leaving the boundary — the same assistive features, with none of the data egress risk.
- **Governed fallback.** Where a governed prediction is rejected — because it fails a confidence threshold or a governance rule — the platform falls back to deterministic behaviour rather than surfacing an ungoverned result.

12.5 The Net Position on AI

Watheeq's use of AI is best summarized as **assistive, governed, metered, and sovereign-capable**. It speeds the reading of contracts and the drafting of instruments; it keeps a human lawyer accountable through mandatory, tracked review; it records and meters its own contribution; it caps its own cost and its own rate; and, for the most sensitive deployments, it can run entirely inside the Kingdom or inside an air-gapped environment with no legal content leaving the customer's control. That combination — real acceleration under real governance — is what makes AI appropriate to a legal system of record rather than a liability within one. It is the difference between "we added AI" and "we added AI that a general counsel can defend."

13. Document Management & E-Archive

13.1 Documents as Governed Legal Assets

Legal work produces documents, and the trustworthiness of those documents — their integrity, their retention, their residency, and their provenance — is a first-order concern for a legal department. Watheeq manages legal documents as governed assets rather than as files in a folder. Every document version carries a content hash; document bytes are held in encrypted object storage; sensitive metadata is encrypted at the field level; and, when a document is archived, its integrity is anchored in a tamper-evident chain that can later be verified. This chapter describes the document and archive capabilities that turn Watheeq's documents into evidence-grade records.

Documents are attached throughout the legal lifecycle — to cases, matters, contracts, consultations, and investigations — and they are versioned. A document version is not merely a new file; it is a distinct, hashed record in a chain, so that the history of a document — every revision, every redline — is preserved and demonstrable. This matters because in legal work the **version** is frequently the point in dispute: which draft was executed, which redline was accepted, what the parties actually agreed. Watheeq's versioned, hashed document model is designed to answer those questions with evidence.

13.2 The 10-Year WORM Legal Archive

The centrepiece of Watheeq's document governance is its e-archiving capability, which writes legal documents into **write-once, read-many (WORM)** storage under records-management controls. This is where a contract, a pleading, or a case file goes to become an immutable record.

The archive supports three retention postures, chosen to match the legal weight of the material:

- **Compliance mode** — the default and strongest posture. An archived object is immutable to **everyone**, including administrators, until its retention period elapses; there is no break-glass override. This is the posture for records that a records-management obligation or a regulator requires to be preserved unaltered.
- **Governance mode** — permits a controlled break-glass reclaim for exceptional, authorized circumstances, so that an organization retains an emergency path where its own governance allows one.
- **A mutable posture** — for material that is not yet under a records-management obligation.

The default retention is ten years, consistent with records-management practice for legal documents, and is tunable per document. Retention is not a soft policy that a careless action can defeat; in compliance mode it is enforced by the storage layer's object-lock so that the immutability is a property of the storage, not merely of the application.

Two further controls make the archive defensible as a legal record store:

- **Legal hold.** An object under legal hold cannot be disposed of even if its retention has elapsed and even under a governance-mode break-glass reclaim — a hold survives every bypass. This is the litigation- and ransomware-safe floor: material subject to a hold is preserved until the hold

is deliberately released. A legal department can therefore place a hold on all material relevant to a dispute and be confident that nothing within it can be destroyed while the hold stands.

- **Tamper-evident manifest.** As documents are archived, their content hashes are chained into an archive manifest — each entry's hash computed over the document identity, version, content hash, object hash, and the previous entry's hash. This is the same hash-chaining pattern the platform uses for its audit trail, so a single verification approach covers both, and the integrity and sequence of archived documents can be independently demonstrated.

Disposal, when it is permitted at all, is deliberately hard: it is triple-gated behind a break-glass authorization, the absence of any active legal hold, and elapsed retention, with the storage layer's object-lock as a defence-in-depth backstop. The design intent is that destroying a legal record is a governed, evidenced, multiply-gated act — never an accident and never a quiet one.

13.3 Backend Flexibility Without Losing Governance

Organizations already have document stores they trust, and Watheeq does not force them to abandon those. The archive is backend-flexible: it can write to a **CMIS** document-management system (such as Alfresco), to an **S3-compatible object store** with object-lock, or to a **Microsoft SharePoint** site. The governance — WORM posture, legal hold, retention, the tamper-evident manifest, and residency — is applied by Watheeq on top of whichever substrate is chosen. Where a backend cannot itself perform a particular control (for example, SharePoint legal holds are managed in Microsoft Purview rather than driven by the connector), Watheeq is honest about the boundary rather than pretending to drive it. The organization gets Watheeq's records governance over the storage it already operates.

13.4 In-Kingdom Residency, Enforced Fail-Closed

The archive enforces Saudi data residency, and it does so **fail-closed**. When in-Kingdom-only operation is required — which is the default — the archive checks the resolved storage region against an explicit allow-list of in-Kingdom regions and refuses to connect or to write to any region outside the Kingdom, **before any bytes are written**, not after the fact. The allow-list explicitly includes sovereign and local Saudi regions and explicitly excludes regions that are outside the Kingdom, including the nearby regional cloud regions that a careless configuration might otherwise select. An out-of-Kingdom target produces a clear, sentinel error and the write is refused.

This is the concrete expression, at the document layer, of Watheeq's sovereignty promise: an organization's archived legal documents cannot be silently written to storage outside Saudi Arabia. It is a control in code, verifiable and testable, not a clause in a data sheet.

13.5 Field-Level Encryption of Sensitive Metadata

Beyond the encrypted object storage of document bytes, Watheeq encrypts the sensitive **fields** of legal records — the personal and confidential attributes of contracts and related documents — using authenticated AES-256-GCM encryption. Encrypted values are marked with a version prefix so the scheme can evolve, and key custody is either held in software or delegated to an external key-management system or vault, so the master key can be governed independently by the security function. In a production deployment this encryption is enforced by policy: the service refuses to

start in a protected profile with field encryption disabled. The effect is that the most sensitive attributes of a legal record are protected at rest independently of, and in addition to, the storage layer's own protection.

14. E-Signature & Identity

14.1 Making an Electronic Signature Legally Meaningful

Executing a legal instrument electronically, in a way that carries evidentiary weight, requires far more than pasting an image of a signature. It requires a confirmed signatory identity, a signature backed by a trusted certificate, and a durable, tamper-resistant record of the whole act. Watheeq integrates the Saudi rails that produce legally-meaningful electronic signatures and orchestrates them so that a Watheeq-executed instrument is non-repudiable. This chapter describes those rails, the crucial separation of identity from signature, the evidence the platform captures, and the honesty with which the platform reports what is live and what is not.

14.2 The Signature and Identity Rails

Watheeq speaks to three distinct Saudi services, each with a specific legal role:

- **emdha — the qualified signature.** emdha is a Trust Service Provider licensed by the Saudi National Digital Certification Center (NCDC). It produces **qualified electronic signatures** backed by a PKI certificate under the Saudi Electronic Transactions Law — the highest evidentiary tier of electronic signature. Every signature Watheeq produces through emdha is recorded with its kind ("qualified") and its legal basis (the Electronic Transactions Law), so the record carries the signature's legal character, not just its bytes.
- **Najiz — the Ministry of Justice portal.** Najiz (accessed through the Takamul integration surface) is the court rail. Watheeq uses it to synchronize hearings, cases, judgments, and enforcement (Tanfeedh) records, and to **issue powers of attorney (wakala)** through the portal — a court-connected legal act rather than a private signature.
- **Nafath — national identity confirmation.** Nafath is the Saudi national single-sign-on identity service. Its role is to confirm **who** a person is, to an assured level — not to sign.

14.3 Identity Is Not Signature — and Watheeq Keeps Them Distinct

A recurring error in electronic-signature systems is to conflate confirming identity with signing. Watheeq keeps them structurally distinct, because in law they are distinct acts: Nafath confirms identity; the Trust Service Provider produces the signature; and a legally-binding electronic execution pairs the two — a confirmed identity, then a qualified signature bound to it.

Watheeq enforces this pairing with real gates:

- **A minimum level of assurance on identity.** Nafath confirmations are graded by strength — from a simple in-app approval, up through an in-app number-match (where the citizen matches a displayed number in the Nafath app), to a biometric confirmation. Watheeq sets its floor at the number-match level: an identity confirmation below that assurance is **not accepted as a valid basis for signing**, even if the citizen "approved," and the gate fails closed. An operator may raise the floor (for example, to biometric) but the platform will not silently lower it. This ensures that the identity behind a signature was confirmed to a strength adequate for a legal act.

- **A wakala cannot be issued without a bound identity.** Issuing a power of attorney through Najiz is hard-gated on a Nafath identity reference: without a confirmed identity to bind it to, the platform refuses to issue the wakala rather than fabricate an unbound authority.

Status and assurance are tracked as independent properties: the identity status answers "did the citizen approve?" while the level of assurance answers "was the approval strong enough to anchor a signature?" A confirmation can be complete yet still fail the assurance gate — which is exactly the discrimination a legal act requires.

14.4 The Evidence Envelope

Every signature carries an **evidence envelope** built for later scrutiny: the provider's envelope and event identifiers, a per-recipient signatory identifier, a delivery status, an evidence hash (typically a SHA-256 digest, a cryptographic fingerprint of the signed artifact), and metadata recording the signature kind, its legal basis, the provider portal, and whether the dispatch was live or sandbox. Inbound provider callbacks — the messages by which the trust service tells Watheeq that a signature completed — are authenticated by HMAC signature over the raw body, with constant-time comparison, before the body is parsed, so a forged callback cannot inject a false completion. The upshot is that a signed instrument in Watheeq is accompanied by a structured, verifiable record of who signed, when, through which provider, to what legal standard, and with what proof.

14.5 Honest Status — Sovereign-Ready, Not Faked

Watheeq is scrupulously honest about the state of these government and trust-service rails. They are built and hardened — the connectors do real OAuth, real mutual-TLS where required, real HMAC verification, and real evidence capture — but they operate against clearly-labelled **sandbox** transports until live credentials are provisioned by the respective authority. A gov-gated connector grades itself "not configured" or "planned" and **never reports a healthy live government connection it does not have**, and a sandbox acknowledgement is explicitly marked and is never presented as a live qualified signature or a live court reconciliation. For a government or enterprise buyer, this honesty is itself a feature: a platform that refuses to fake a live court connection is more trustworthy than one that claims blanket readiness. Bringing these rails live is an onboarding and credentialing exercise against the authorities — the build is done.

15. Calendar, Deadlines & KSA Localization

15.1 Deadlines Only Mean Something on the Right Calendar

A legal deadline computed on the wrong calendar is worse than no deadline at all, because it invites false confidence. Watheeq computes every service-level deadline, acknowledgement window, and escalation rung against a genuine Saudi working-time profile — the working week, public holidays, and a Ramadan working-hours overlay — in the Kingdom's timezone. "Two working days" therefore means two Saudi working days, and a deadline never silently lands on a Saudi weekend or a public holiday. This is the difference between a legal-ops platform that was localized for Saudi Arabia and one that merely had its labels translated.

15.2 The Saudi Working Calendar and Holidays

The working calendar encodes the Saudi working week and the Kingdom's public holidays. Holidays include both **fixed-Gregorian national days** — Founding Day (22 February) and National Day (23 September) — and **Hijri religious holidays**, the two Eids (Eid al-Fitr and Eid al-Adha), whose Gregorian dates move each year with the lunar calendar. A **Ramadan overlay** adjusts working hours for the holy month. Administrators maintain working calendars and their holidays, so an organization can reflect its own observed calendar, including any organization-specific closures, on top of the national baseline. Every SLA acknowledgement, turnaround, and escalation deadline is materialized against this calendar, in the Asia/Riyadh timezone.

15.3 Hijri Dates, Arabic Numerals, and the Riyal

Watheeq presents dates in the official **Umm al-Qura Hijri calendar** alongside Gregorian — the civil Hijri calendar the Kingdom actually uses — so a legal user sees dates in the calendar they work in, with the Gregorian equivalent alongside where useful. In Arabic mode, numbers render in **Arabic-Indic digits** and currency renders in **Saudi Riyal** with the correct Arabic conventions and symbol. These are not per-screen implementations that drift apart over time; they are delivered through a single localization layer that every legal domain consumes. A developer building a new legal screen does not re-implement Hijri formatting — they consume the shared layer, which reads the active locale and dispatches Arabic or English automatically, so that Hijri-aware, Arabic-correct, Riyal-first output is the default everywhere. The layer is built to be safe under server-side rendering, avoiding the subtle bugs that afflict date and locale handling in web applications.

15.4 Right-to-Left by Default, Bilingual by Construction

The application defaults to Arabic and right-to-left layout, and every role, service-catalog step, workflow form field, connector configuration field, and screen carries first-class Arabic alongside English. Bilingual, RTL-correct presentation is therefore the norm rather than an afterthought, and it runs end to end — from the fourteen legal roles' Arabic names, through the eight-service catalog, to the analytics labels and the report headings. For a Saudi legal department, the platform reads as a native Saudi system, not a foreign product with an Arabic pack bolted on.

15.5 The Calendar as an Active Engine

On this foundation, the calendar is not a passive display but an active engine that drives the department's time-critical obligations: contract expiry and renewal monitoring with reminders as key dates approach; obligation due-date tracking; hearing schedules; and SLA acknowledgement and escalation reminders — all generated against Saudi working time and delivered through email, calendar, and in-app channels, with deduplication so the same reminder is never sent twice. The department's calendar watches its deadlines and prompts action before they slip, on the Kingdom's own calendar.

16. Analytics, Dashboards & Reporting

16.1 Turning Governed Data into Management Insight

Because Watheeq holds the department's work as governed, structured data — every request on the spine, every case and contract with status and dates, every SLA clock — it can turn that data into genuine management insight. The analytics layer is not a separately maintained spreadsheet that drifts from reality; it measures the real, audited state of the legal function. For a general counsel, that is the difference between a dashboard that looks impressive and one that can be relied upon in front of the board.

16.2 Headline KPIs

Watheeq presents a consolidated legal affairs dashboard and dedicated analytics surfaces with the measures leadership actually watches, rendered as scannable KPI tiles with inline trend sparklines. For legal operations, the headlines include active and recently closed matter counts, average cycle time to close, settlement cycle time, weekly throughput, and the busiest handling officer. For portfolio risk, they include total portfolio value and active value, value at risk (with a trend), high-risk share, the count of contracts expiring within ninety days, and an average risk score. Every one of these is presented bilingually and RTL correctly, with Hijri aware date labels and Riyal first values, through the same localization layer as the rest of the suite.

16.3 Operational and Risk Analytics

Beyond headline numbers, Watheeq provides a rich set of visual analytics:

- **SLA compliance** measured per quarter against a target rate, with both an outcome breakdown (on-time, breached, pending) and a trend against the target.
- **Case analytics** — status distributions, litigation posture (plaintiff versus defendant), and case-type breakdowns.
- **Contract analytics** — a lifecycle funnel from draft through active stages with drop-off, review turnaround times, and breakdowns by department, type, and status.
- **Workload analytics** — an officer-by-practice-area workload heatmap and a department-by-domain heatmap, so managers can see where the load falls.
- **Efficiency gauges** — closed-case ratio, approved-contract ratio, SLA on-time rate, and duration adherence.
- **Portfolio risk** — a risk distribution across high/medium/low bands, a value at risk view, and a twelve-month renewal cliff timeline that highlights the month of peak contract expiry exposure so the department can plan ahead.

Filters let leadership slice these by department, status, type, and date range, and **saved views** let a user preserve a particular configuration and return to it, synchronized across their browser sessions.

16.4 The Analytics Catalog

To make the breadth concrete, the following is the visual analytics catalog a legal leadership team has available, each rendered bilingually and RTL correctly:

Visualization	What it answers
SLA outcome chart	How much work was on-time, breached, and pending, per quarter
SLA trend chart	Whether the SLA compliance rate is tracking above or below target over time
Case-status donut	The distribution of cases across their lifecycle states
Litigation posture chart	The plaintiff-versus-defendant split of the litigation portfolio
Contract funnel	Where contracts sit and drop off across the lifecycle from draft to active
Matter-type treemap	The composition of the matter/case portfolio by type, sized by count
Department × domain heatmap	Where load concentrates across departments and legal domains
Officer × practice-area workload heatmap	How work is distributed across handling officers
Efficiency gauges	Closed-case ratio, approved-contract ratio, SLA on-time rate, duration adherence
Turnaround chart	Contract-review, consultation, and request-processing times
Risk distribution & gauge	The portfolio's spread across high/medium/low risk bands
Value-at-risk & renewal-cliff timeline	Financial exposure and the twelve-month expiry profile with its peak month
KPI strips with sparklines	The headline measures with inline trend lines

These are deliberately implemented for performance and correctness — the heat maps and gauges are rendered as lightweight visuals rather than heavy chart components, and all colours flow from the design-system tokens so the entire analytics layer re-themes cleanly, including for dark mode, without per-chart rework. The point of cataloguing them here is not the individual pictures but the breadth: a general counsel has, out of the box, the operational, portfolio, risk, and financial views needed to run the function and to report on it upward.

16.5 The Reporting Surface Underneath

Underpinning the visual layer is a set of governed reporting endpoints, each read-only and permission-gated on a dedicated reporting permission, covering the department's principal reporting needs:

- **Case reporting** — totals and breakdowns by type, department, status, and company-side (plaintiff/defendant), plus closed and under-procedure counts.
- **Contract analytics** — totals, average review turnaround (refined from working-duration facts so the average reflects true working time), and breakdowns by department, type, and status, with the sample size behind each average.
- **Consultation reporting** — totals, breakdowns, and average completion time.
- **Performance scorecard** — average request-processing time, closed-case ratio, approved-contract ratio, overdue-request count, and estimated duration adherence.
- **SLA compliance** — the flagship measure: per-quarter received, on-time, breached, and pending counts, the resulting rate against a ninety-percent target, and whether each quarter and the portfolio overall met that target.
- **The consolidated legal-affairs dashboard** — a single fan-out that assembles all of the above plus the current quarter's SLA rate, cached briefly for responsiveness.

Every report echoes its filters back in the response for transparency, and each is exportable to CSV and Excel for circulation and board reporting.

16.6 Governed, Read-Only Reporting and Export

Underpinning the dashboards is a read-oriented reporting service that aggregates from the real source tables — cases, contracts, consultations, requests, and SLA clocks — and refines its working-duration averages from event-sourced duration facts so that turnaround figures reflect true *working* time rather than raw elapsed time. The reports cover cases, contracts, consultations, and performance, and culminate in the flagship quarterly **SLA-compliance** measure against a ninety-percent target. Reports carry their filters back in the response for transparency, and they export to **CSV and Excel** for circulation and board reporting. Crucially, all reporting is read-only and permission-gated on a dedicated reporting permission, so analytics can never become a side channel to alter a legal record.

16.5 Why This Matters to the Business

Because the analytics rest on the same governed record described throughout this document, they are trustworthy by construction. When a Watheeq dashboard reports that the department met its SLA target this quarter, that number is computed from the real, audited SLA clocks — the same clocks that drove the real acknowledgements and escalations — on the Saudi working calendar. A general counsel can put those figures in front of a board or a regulator and stand behind them, because they are not a presentation layer over guesswork; they are a measurement of the department's actual, governed operation.

17. Roles, Personas & Access

17.1 A Legal Department Modelled Faithfully

WatheeQ's access model is built around the real organizational shape of a corporate legal department rather than a generic administrator and user split. It recognizes **fourteen legal roles** spanning four tiers — the requesting business, the legal function, an oversight layer, and system administration — and gives each role its own landing experience, its own scoped navigation, and its own precisely bounded set of permissions. This is the backbone around which the whole suite is organized: what a user can see and do is a direct, faithful projection of their place in the legal organization. The roles are defined in code as the authoritative source, seeded into each tenant, and — importantly — the seeding is **asserted**, meaning the platform will fail its start-up readiness check if the full set of fourteen roles and their segregation of duties exclusions are not present for a tenant. There is no silent fallback to a coarse "everyone can write" model.

17.2 The Fourteen Roles

Tier	Role (English)	Role (Arabic)	Escalation	What they do
Business	Requester / Employee	الموظف / مقدم الطلب	—	Raises legal service requests they are eligible for
Business	Department Manager (Requesting)	مدير الإدارة الطالبة	L2	Requester side delegation of authority approver
Business	Business Unit CEO	الرئيس التنفيذي للقطاع	—	High value approver within their own business unit
Business	CEO / Executive Management	الرئيس التنفيذي للشركة	—	Issues the directive to commence legal action
Legal	Legal Director	مدير الإدارة القانونية	—	Top legal authority; full operational + configuration authority
Legal	Cases & Investigations Section Manager	مدير قسم القضايا والتحقيقات	—	Runs cases/investigations ; can assign, approve, close
Legal	Contracts Section Manager	مدير قسم العقود	—	Runs contract review; holds final contract sign-off
Legal	Case Supervisor	مشرف القضايا	L1	First tier case review; no assign or close
Legal	Contracts Supervisor	مشرف العقود	—	Distributes and first tier reviews contracts; no approve/close
Legal	Legal Officer / Handling Lawyer	الموظف المختص / المحامي	—	Direct handler — pleadings, memos, hearings; drafts, no

				approve/close
Legal	Legal Advisor / Consultant	المستشار القانوني	—	Reviews contracts (recommends only), answers consultations
Oversight	Shared Services Unit Manager	مدير وحدة الخدمات المشتركة	L3	System owner oversight; tenant-wide read + audit
Oversight	Auditor / Compliance Officer	المدقق / مسؤول الالتزام	—	Read-only everywhere; changes nothing
Admin	System Administrator	مسؤول النظام	—	Configures the platform; makes no legal decisions

Several design choices in this matrix are worth drawing out because they encode real governance:

- **The legal director** holds full operational authority across every legal domain — including the two restricted allocation verbs, assign (for cases) and distribute (for contracts) — plus the configuration authorities for service catalog, SLA, and escalation, and read access to audit and integrations. This role is the department's apex, and its breadth is deliberate.
- **Officers draft; managers approve.** A legal officer can create and edit cases, investigations, and settlements, but holds no approve, close, or assign verb. A case supervisor can review and approve at the first tier but cannot assign work or close a case. Only the cases section manager can assign, approve, *and* close. The same pattern applies to contracts, where a contracts supervisor distributes and first-tier reviews but only the contracts section manager gives final sign-off. This encodes the tiered review a legal department actually runs.
- **The auditor sees everything and changes nothing.** The compliance auditor holds view and read permissions across every domain and read access to the audit trail, and holds *no* mutating verb anywhere — no add, edit, approve, close, assign, or distribute, and not even the coarse write permission. This is a deliberate segregation of duties safeguard so that oversight of the legal function cannot become a channel to alter it.
- **The system administrator runs the platform but makes no legal decisions.** The administrator holds the configuration authorities — catalog, SLA, escalation, notifications, integrations, security, and role administration (itself constrained by anti-escalation rules so the administrator cannot grant themselves operational authority) — but holds *no* operational legal verb. The person who runs the system is structurally separated from the people who make legal decisions in it.

17.3 Least Privilege and Independent Verbs

Access is granted at the granularity of a domain and a verb. Each legal domain — request, case, investigation, settlement, consultation, contract, document — and each configuration domain — catalog, SLA, escalation, notification, role, integration, security, audit — has its own verbs: view, add, edit, approve, close, assign, distribute, and manage as applicable. A role holds only the verbs its function requires.

Two principles make this precise rather than approximate. First, **there is no implicit escalation between verbs**: holding edit does not confer approve; holding approve does not confer close; the permission expansion is one-directional (an operational verb implies view, and a configuration manage implies the lower verbs of that configuration domain, but nothing implies a *higher* operational authority). Second, **the elevated verbs — approve, close, assign, distribute, manage — accept no coarse fallback**. A user holding a broad write permission is explicitly *denied* on these sensitive routes; only the exact granular permission, or an appropriate wildcard, passes. This closes the common and dangerous gap where a "power-user" write permission silently confers approval authority. The view, add, and edit routes retain a coarse fallback for backward compatibility, but the routes where authority is actually exercised do not.

17.4 Role-Aware Experience

A user's experience is shaped by their role from login onward. Each role lands, on sign-in, on the workspace most relevant to it — the requester on the service desk, the director on a command view, the officer on their work queue, the auditor on compliance, the administrator on the admin console. Navigation is filtered to what the user may access, evaluated against their true, fine-grained permissions, so a role never sees menu entries for capabilities it lacks, and navigation groups that would be empty are dropped rather than shown hollow. Actions within a screen — the approve button, the assign control, the close action — are gated on the same granular permissions and capability flags, so the interface offers a user only the actions they can actually take.

Where a user holds more than one legal role, a **persona switcher** lets them move between personas; on switching, the platform re-lands them on the appropriate workspace and re-scopes their navigation and actions. Underpinning this is a "who am I, in legal terms" endpoint that returns the user's active and available legal roles, their granular effective permissions, their capability flags, and their recommended landing — and the front end unions those granular permissions into its permission checks. This hydration is load-bearing: the authentication token carries only role slugs, and it is the effective-permissions response that gives the interface the fine-grained truth of what the user can do, rather than a coarse approximation. The whole experience is bilingual and RTL-correct: every role carries an Arabic name, and the persona chrome, navigation, and capability views render correctly in Arabic. A Saudi legal department sees itself — in its own language and its own structure — in the software.

18. Integration Platform

18.1 A Uniform Way to Connect the Department

A legal department does not operate in isolation. It authenticates against the enterprise identity provider, provisions users from HR, archives to the corporate document store, sends and receives email, signs through trust services, and connects to the Saudi courts and identity rails. Watheeq provides a **uniform integration framework** for all of this: one consistent way to configure, test, health-grade, and operate a connection, with consistent secret custody and consistent honesty about what is live and what is not.

Every connector is built on a common adapter base — it knows its kind and can probe its own health — and grows optional, well-defined capabilities as appropriate: the ability to **test a connection** without side effects, to **synchronize** data (full, delta, or preview), and to **invoke** actions. This capability model means the framework can treat a wide range of external systems uniformly while each connector exposes exactly the operations it genuinely supports.

The framework provides the cross-cutting concerns a legal department needs from any integration:

- **Secret custody.** Configuration secrets are encrypted at rest and are never logged or echoed; any surface that displays a connector's configuration redacts the secret fields behind a sentinel value. Each connector resolves its plaintext configuration through a field-decrypting repository, not through the redacting display layer, so secrets are available to the connector at run time but never leak to a screen or a log.
- **Health honesty.** Each connector reports a health grade derived from its real configuration and, where appropriate, a real reachability probe — and it never fabricates a healthy state it does not have.
- **Resilience and control.** The framework includes circuit breaking, a dead-letter path, a sync ledger, and health history, plus an **egress policy** that can constrain both the **fields** a connector may send outward (for data minimization) and the **regions** it may send them to (for residency) — failing closed when a field or region is not permitted.
- **A schema-driven admin console.** An administration console renders each connector's configuration form dynamically from a bilingual schema, and offers Test Connection, Sync Now, and health views, gated on dedicated integration-read and integration-manage permissions.

18.2 The Connectors — Production-Ready, Self-Serve

A first group of connectors is production-ready and self-serve: an organization can configure and use them directly.

- **Single sign-on (OIDC / SAML).** Wires the department to enterprise identity providers such as Microsoft Entra, Okta, and Keycloak. Its Test Connection performs real work — live OIDC discovery and key-set fetch (asserting at least one signing key and a client identifier), or live SAML metadata fetch and parse (asserting a sign-on URL and an unexpired signing certificate) — so a "healthy" grade means the identity provider genuinely responded.

- **HR / identity provisioning.** Pulls upstream users, groups, and workers and reconciles them into the organizational registry, over SCIM, HRIS APIs (Workday, SuccessFactors, Oracle HCM shapes), or file- and directory-based transports. Full, delta, and dry-run preview sync modes are supported. The Saudi tier-two government HR sources (the labour and residency systems) are honestly gov-gated: configurable but not wired to a live API until government onboarding, and never returning a fabricated roster.
- **Inbound SCIM 2.0 server.** The push counterpart to the HR pull: an external identity provider can provision users and groups *into* Watheeq over a standards-compliant SCIM endpoint, authenticated per tenant by a hashed bearer token, provisioning the same organizational-registry targets. Deactivations are reversible soft-deactivations, never hard deletes.
- **E-archiving.** Writes legal documents to CMIS, S3-object-lock, or SharePoint under the WORM, legal-hold, retention, manifest, and in-Kingdom-residency governance described in Chapter 13.
- **Email.** Unifies inbound intake (HMAC-verified mailbox) and outbound dispatch over SMTP, Amazon SES (with inline request signing), Microsoft Graph, or webhook. A "both-direction" endpoint is graded healthy only when both legs verify.
- **Generic internal REST / webhook.** A signed, authenticated, egress-fenced catch-all any tenant can point at its own internal systems, with an allow-list of permitted outbound hosts that fails closed, HMAC-signed outbound requests, and HMAC-verified inbound webhooks with timestamp-tolerance replay protection.

18.3 The Connectors — Gov-Gated, Sovereign-Ready

A second group is built and hardened but operates against clearly-labelled sandbox transports until live government credentials are provisioned:

- **E-signature**, including the qualified trust-service (emdha) and Ministry of Justice (Najiz) signing rails.
- **Najiz** court-portal synchronization — hearings, cases, judgments, enforcement — and the issuance of powers of attorney, gated on confirmed identity.
- **Nafath** identity confirmation, with an enforced minimum level of assurance.

Every gov-gated connector states its status plainly: it grades itself "not configured" or "planned," it labels its sandbox transports explicitly, and it **never reports a healthy live government connection it does not have**. In production mode each does real work — real OAuth client-credentials, real mutual-TLS where required, real HMAC verification — so bringing them live is a matter of provisioning credentials against the authorities, not of building the integration. For a government or enterprise buyer, this honesty is a differentiator in its own right.

19. Security, Compliance & Data Residency

19.1 Tenant Isolation

Watheeq is multi-tenant, and every customer's data is isolated. Isolation is enforced deterministically at the query layer: the tenant identity is carried in the signed authentication token, injected by middleware into the request context, and threaded as the first parameter of every service and repository call, so that every data access is filtered by tenant. A user authenticated to one tenant cannot retrieve another tenant's records — a request for a record outside the user's tenant simply returns nothing, which the platform surfaces as "not found," indistinguishable from the record genuinely not existing. In a dedicated on-premise or air-gapped deployment there is only one tenant, so the concern is moot in a good way; in the SaaS model, this is the guarantee that a legal department's records are its own.

19.2 Layered Access Control

On top of tenant isolation sit multiple, composable access layers:

- **Role-based permissions** at domain and verb granularity, with independent verbs and no coarse fallback on the elevated actions (Chapter 17).
- **Organizational scoping**, which can confine access not only by role but by the organizational entity a user belongs to and its hierarchy — so a supervisor sees their section's work, not the whole department — enforced per entity on the sensitive verbs.
- **An attribute-based policy layer**, which can add tenant-scoped attribute policies on top of role checks.
- **Dynamic segregation of duties**, which prevents an author from approving or closing their own record and requires distinct approvers across review rounds, failing closed and applying to everyone including administrators.

These layers compose: a sensitive action must pass the permission gate, the organizational scope gate, and the distinct actor gate. Access is therefore defined by the intersection of **who you are**, **where you sit in the organization**, and **whether you are conflicted on this record** — a far stronger model than a single role check.

19.3 Encryption and Tamper-Evidence

Sensitive legal data is encrypted at rest with authenticated AES-256-GCM encryption, with key custody either in software or delegated to an external key management system or vault, and encryption is enforced by policy in protected deployments — the service refuses to run in a protected profile with sensitive field encryption disabled. Traffic is protected in transit: the sensitive service-to-service paths (such as the disaster recovery and security telemetry planes) use mutual TLS with a minimum TLS 1.2 floor and certificate pinning, and the public edge enforces HTTPS with strict transport security. And every material action is recorded in a **tamper-evident, hash-chained audit trail** written to an append-only, immutable store whose mutation is blocked at the database layer — so the record of who did what, when, and under whose authority cannot be silently altered, and its integrity can be independently verified by recomputing the chain.

19.4 Saudi Data Residency, Enforced in Code

The platform's residency posture is its signature sovereignty feature, and it is enforced in code rather than promised on paper, at two layers:

- **A request-time control** checks each tenant's residency binding on inbound requests and refuses to serve data across a disallowed region, failing closed if it cannot verify the binding. Every denial is recorded as a structured "residency denied" audit event carrying the tenant, the tenant region, the service region, and the reason.
- **A data-plane control** refuses to **write** legal records and archives to storage outside the Kingdom when in-Kingdom operation is required, using an explicit allow-list of in-Kingdom regions and an explicit exclusion of out-of-Kingdom ones, and it too fails closed — rejecting an out-of-Kingdom target **before any bytes are written**. The control-plane and data-plane decisions share the same residency logic and the same tenant-region source, so they cannot drift apart.

This is the concrete substance behind the word "sovereign": an organization's most sensitive legal records are kept in the Kingdom by an enforced, audited technical control — not by trust, and not by a promise that could be quietly broken by a misconfiguration.

19.5 Compliance Posture

WatheeQ is built to align with the Kingdom's Personal Data Protection Law and the control families of the national cybersecurity framework and the internationally recognized information-security standards. That alignment is expressed as concrete controls — encryption at rest and in transit, in-Kingdom residency, fine-grained access control and segregation of duties, data-minimizing egress policy, and a tamper-evident audit trail — and, distinctively, it produces **evidence**: the platform can demonstrate who accessed what, who approved what under whose authority, that records were not altered, and that data stayed in the Kingdom. For an organization that expects to be audited or to defend its conduct before a regulator, this evidentiary capability is often worth more than any single certificate. (The compliance posture is described in full, with the honest distinction between designed-for control coverage and formal certification, in the companion legal-and-compliance document.)

20. Deployment, Administration & Roadmap

20.1 Three Deployment Models

Watheeq meets a Saudi legal department wherever its security posture requires it to be, with the same product, governance, and Arabic-native experience across all three:

- **SaaS (in-Kingdom multi-tenant).** Hosted within the Kingdom and consumed as a service, with logical isolation from other tenants. This is the fastest path to value and shifts most operational responsibility to the provider.
- **On-premise / VPC.** The platform runs in the organization's own data centre or private cloud — the common choice for large enterprises and government bodies that require infrastructure control. Here the customer bears more of the operational and physical-security responsibility.
- **Air-gapped.** The platform runs in a fully isolated environment for the most sensitive engagements, with self-hosted AI so that legal content and AI processing never leave the boundary, and with an appropriately isolated handling of the government integrations that by nature reach external services.

The deployment model changes *where* the platform runs and *who* bears the operational load — not *what* it does.

20.2 Watheeq as a Stand-Alone Suite

Watheeq can be licensed and deployed as a stand-alone legal product. A Watheeq-only tenant runs the legal service plus the shared platform core — identity and authentication, the workflow engine, notifications, file storage, and the immutable audit ledger — which is a materially smaller footprint than the full multi-suite platform (on the order of a third of the compute). The platform supports self-serve onboarding of a legal-only tenant whose licence is scoped to the legal suite, and it automatically provisions that tenant with the Legal Affairs starter configuration: the eight-service catalog, the organizational registry, the working calendar, the SLA targets, and the fourteen-role matrix with its segregation-of-duties exclusions. A new legal customer is therefore stood up on a working, governed legal department — not an empty shell that must be configured from scratch before it can do anything.

20.3 Administration

The system administrator governs the platform's configuration without touching legal decisions. The administrative surface covers the service catalog, working calendars and holidays, SLA targets, escalation ladders, attachment policies, the fourteen-role matrix (constrained by anti-escalation rules), integrations, and security settings. Administration is itself audited, and the most sensitive administrative acts can be gated by approval — for example, promoting a workflow definition to production can require a distinct approver — so that configuring a legal platform is a governed act, consistent with the governance the platform applies to legal work itself. The clean separation between the administrator (who configures) and the legal roles (who decide) means an organization can delegate platform administration to IT without granting any legal authority.

20.4 Roadmap

Watheeq is built and hardened as a legal operations platform today, and its forward direction is clear and stated honestly. The most consequential items on the path ahead are:

- **Government go-live.** Bringing the Najiz, Nafath, and emdha rails from their built-and-hardened sandbox state to live production against real government credentials. This is an onboarding and access exercise against the authorities, not a build exercise — the integrations are complete and hardened.
- **Tighter recovery objectives.** Continuing to tighten the platform's disaster-recovery objectives toward the more aggressive recovery-point and recovery-time targets on its roadmap.
- **Ordinary product maturation.** Deeper analytics, broader connector coverage, and continued refinement of the Arabic-native experience.

The honest framing that runs through this document applies to the roadmap too: Watheeq is a real, governed, sovereign legal platform now, with a defined and credible path to bringing its Saudi government integrations fully live.

20.5 Capability Summary Matrix

The following matrix consolidates the platform's principal capabilities against the chapters that describe them, as a single reference for a reader who wants the whole surface at a glance. Status reflects the honest posture described throughout: *Live* means built and operational; *Sovereign-ready* means built and hardened but operating against sandbox transports pending government credentialing.

Capability area	Principal capabilities	Chapter	Status
Legal intake	Platform submission, HMAC-verified email intake, eligibility gate, urgency justification, request spine	2–3	Live
Case management	Two-phase litigation intake, plaintiff/defendant, strength, parties, hearings, tasks, assignment verbs	3, 8	Live
Matter management	General legal-work container, conflict check, triage, sub-resources, settlement anchor	3, 9	Live
Investigations	Investigation lifecycle, parties/evidence, SLA facts, approval chain	4	Live
Consultations	Consultation intake, classification, routing, response, approval, archive	5	Live
Legal requests & catalog	Eight-service catalog, request FSM, auto-spawn routing, priority model	2, 6	Live

Contract lifecycle	Review desk, drafting, clause library, redline, approval, repository	7	Live
Litigation & hearings	Pleadings, hearings, judgments (idempotent), statement of claim drafting	8	Live
Settlements	Proposal→negotiation→approval→execution, ADR methods, matter close	9	Live
Approvals & policy	Versioned/audited/conflict checked approval policies, quorum, recommendation	10	Live
Delegation of authority	X.509/PKI validated approvals, amount binding, strict/fallback modes	10	Live (PKI provisioning per deployment)
Segregation of duties	Static role exclusions, dynamic author≠approver, no coarse fallback	10, 17	Live
Workflow engine	FSM engine, human tasks, incidents, BPMN, promotion, expressions	11	Live
AI assistance	Clause/obligation extraction, drafting, governance, metering, local models	12	Live
Document & e-archive	Versioned docs, 10-yr WORM, legal hold, manifest, backend flexibility	13	Live
Field encryption	AES-256 GCM sensitive fields, software/external key custody, enforced	13, 19	Live
E-signature	Qualified TSP (emdha), MOJ (Najiz), evidence envelope	14	Sovereign ready
Identity	Nafath confirmation, minimum level of assurance, identity≠signature	14	Sovereign ready
Court integration	Najiz hearing/case/judgment sync, powers of attorney	14, 18	Sovereign ready
Calendar & localization	Umm al-Qura Hijri, Arabic-Indic, SAR, holidays, Ramadan, RTL	15	Live
Deadlines & SLA	Working calendar SLA clocks, acknowledgement, three-rung escalation	3, 15	Live
Analytics & reporting	KPI strips, operational/risk analytics, CSV/XLSX export, saved views	16	Live
Roles & personas	14-role matrix, role-aware landing,	17	Live

	persona switch, effective permissions		
Integration platform	Connector framework, SSO/HR/SCIM/e ² archive/ email/internal, admin console	18	Live (self ² serve) / Sovereign ² ready (gov)
Security & residency	Tenant isolation, layered access, encryption, tamper ² evident audit, in ² Kingdom residency	19	Live
Deployment	SaaS / on ² premise / air ² gapped, stand ² alone Watheeq onboarding	20	Live

20.6 Closing

Across twenty chapters, this document has described a legal operations platform built specifically for a Saudi legal department: one governed system for the whole legal lifecycle, from a controlled intake through authorized, evidenced approvals to an immutable, in²Kingdom archive; a shared, reliable engine driving every workflow; genuine AI assistance kept under real governance; qualified electronic signatures and confirmed identity through the Saudi rails; and an experience that is Arabic²native and Hijri²aware by construction rather than by translation. The recurring theme is sovereignty made concrete — residency enforced in code, records made immutable, authority made provable, and government integration built honestly. That is what distinguishes Watheeq from a generic legal tool re²skinned for the region: it is a legal platform designed, from the data model up, for the Kingdom.